

GamutX

Bat Files

PYTHON SCRIPTING

Recap of Python CLI Tools

- Overview of CLI Tools
- Using `sys.argv`

Overview of CLI Tools

◆ Terminal

- A program that allows users to type and run text-based commands.
- Provides an interface to interact directly with the operating system.
- Examples include: Command Prompt (Windows), Terminal (macOS/Linux), PowerShell, Git Bash.

◆ Python CLI Tools

- Python scripts that can be executed from the command line.
- They often take arguments or parameters to decide what action to perform.
- Useful for tasks like running utilities, automating workflows, or processing data.

Using sys.argv

Let's look at how we can access those **command-line inputs** in **Python**. The most basic way is by using **sys.argv**.

- **sys.argv** is a list in Python that contains the arguments passed to the script.
- The elements are:
 - **sys.argv[0]** → the name of the script.
 - **sys.argv[1:]** → the arguments provided by the user.

Example Command

- If we run this in the terminal:

```
python my_script.py arg1 arg2
```



- Then **sys.argv** will look like this:

```
['my_script.py', 'arg1', 'arg2']
```

So, **sys.argv[1]** would be 'arg1', and **sys.argv[2]** would be 'arg2'.

Basic Example

Let's look at a small script to understand how we use **sys.argv** in practice:

```
import sys
if len(sys.argv) != 3:
    print("Usage: python my_script.py <arg1> <arg2>")
else:
    arg1 = sys.argv[1]
    arg2 = sys.argv[2]
    print(f"Argument 1: {arg1}, Argument 2: {arg2}")
```

checks how many Args passed

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\pralhad\Desktop\test> python script.py hello world
  Argument 1: hello, Argument 2: world
○ PS C:\Users\pralhad\Desktop\test>
```

Key takeaway

- **CLI tools** let you interact with programs directly from the terminal.
- **Python CLI tools** are just Python scripts that can be run from the command line.
- **sys.argv** gives your script the ability to accept arguments:
 - **sys.argv[0]** → script name
 - **sys.argv[1:]** → user-provided arguments
- This makes Python scripts **flexible** and **reusable**

Introduction to .bat Files

- What is a .bat File?
- .bat file in Windows

What is a .bat File?

- A **batch file** is a text file with the extension **.bat** or **.cmd**.
- Used in **Windows** to run a list of command-line instructions automatically.
- **Purpose:** Save time by automating repetitive steps.

Analogy:

"Think of a .bat file as a to-do list where each command is executed in order."



"Windows runs
each step in order"

Command Prompt

```
Step 1- Launch Terminal  
Step 2- run script in terminal  
Step 3- close terminal
```

.bat file in Windows

Let's say you often do the following:

- Open Command Prompt
- Navigate to a folder with your personal scripts
- Run a Python script that greets.

Instead of repeating this manually, you can automate it with a **.bat** file.

daily_greet.bat

@echo off

cd C:\Users\pralhad\Desktop\test

python greet.py

pause

Hides the command output

Runs the greet script

Changes directory to where script is stored

Output

```
C:\WINDOWS\system32\cmd.  X  +  v
Hello user !!
Press any key to continue . . . |
```

What Happen When we Run File?

- When we **double-click** a **.bat** file:
 - a. Windows **opens the Command Prompt**.
 - b. It **executes** each command inside the file, **line by line**.
- This is **useful when we have to repeat the same steps often**.

Manual workflow (without .bat):

- Open the Command Prompt.
- Navigate to a project folder.
- Run a program or script.

Automated workflow (with .bat):

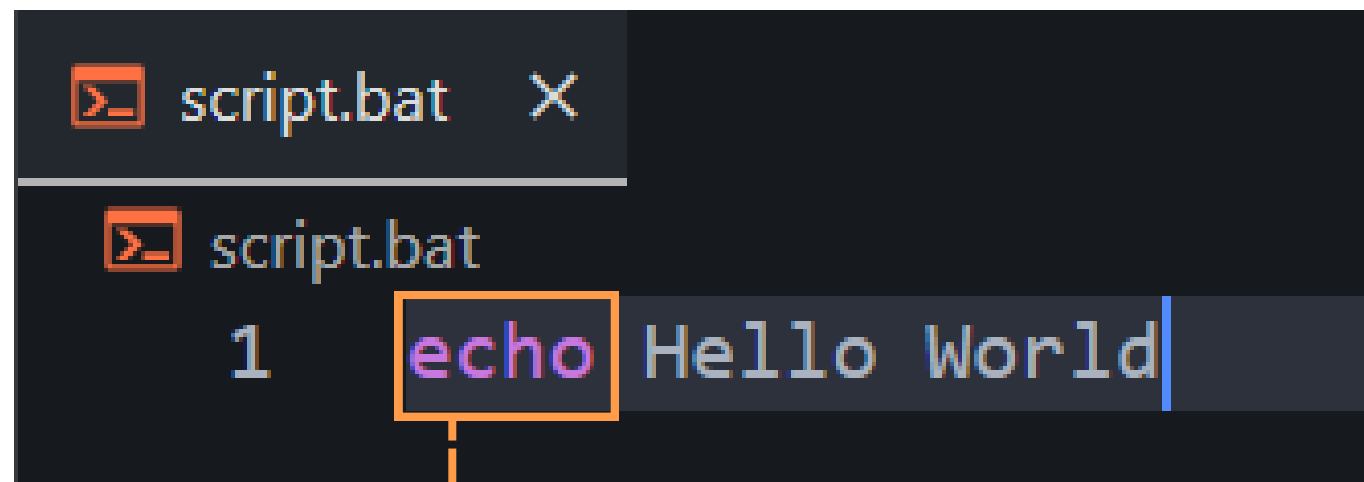
- You double-click the **.bat** file.
- All three steps run automatically, in order.

Getting Started with .bat Files

- Creating a .bat File
- Running a .bat File
- Why Use These Methods?
- Why Does the Window Close Immediately?
- How to Keep the Terminal Open?
- Writing Multiple Commands in a Batch File
- Clean Up the Output with @echo off
- Using the timeout Command

Creating a .bat File

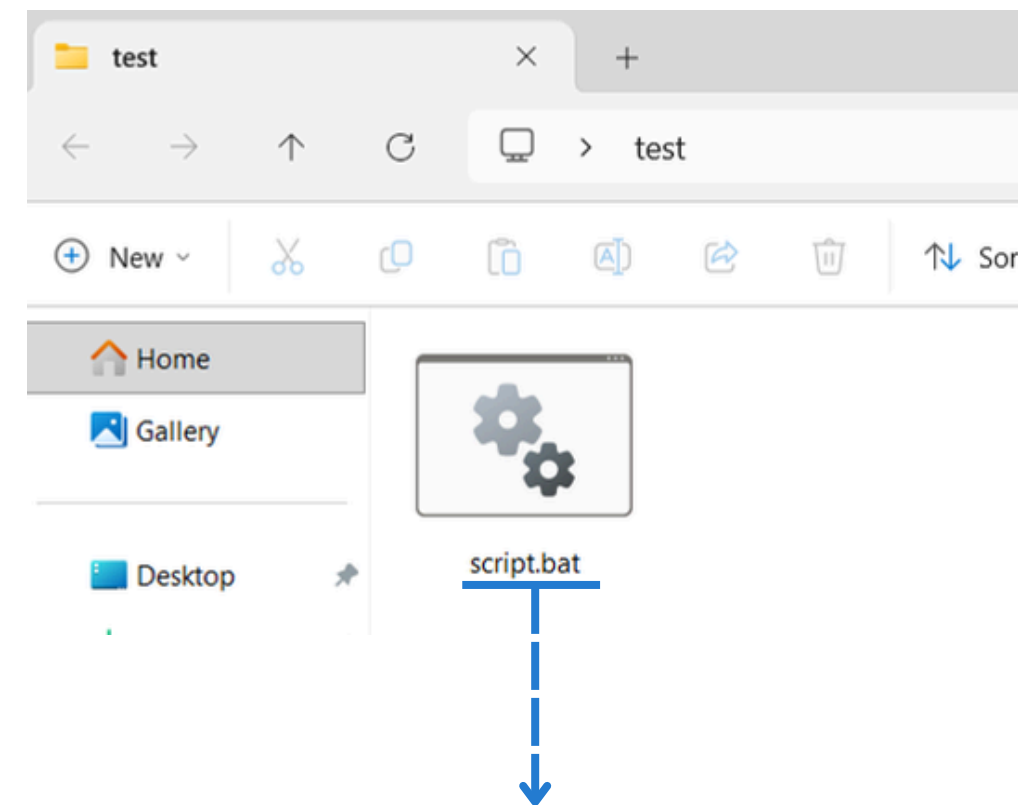
- Open Notepad or vs code.
- Type the following command:



```
script.bat X
script.bat
1 echo Hello World
```

displays text in Command Prompt.

Similar to Python's `print()`



Make sure the file extension is **.bat**

There are
Two ways to run **.bat file**

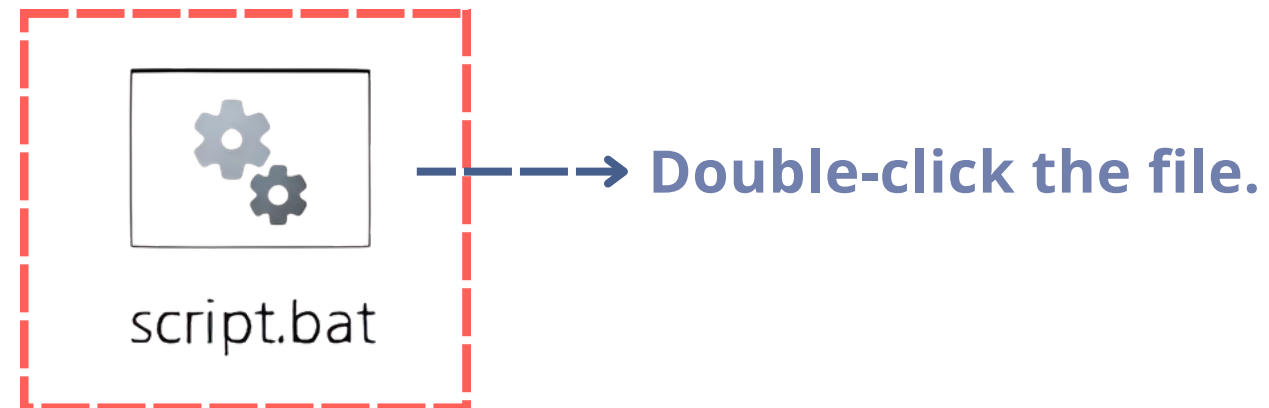
By double-clicking the file

By using the Command Prompt

Method 1: Run by Double-Clicking

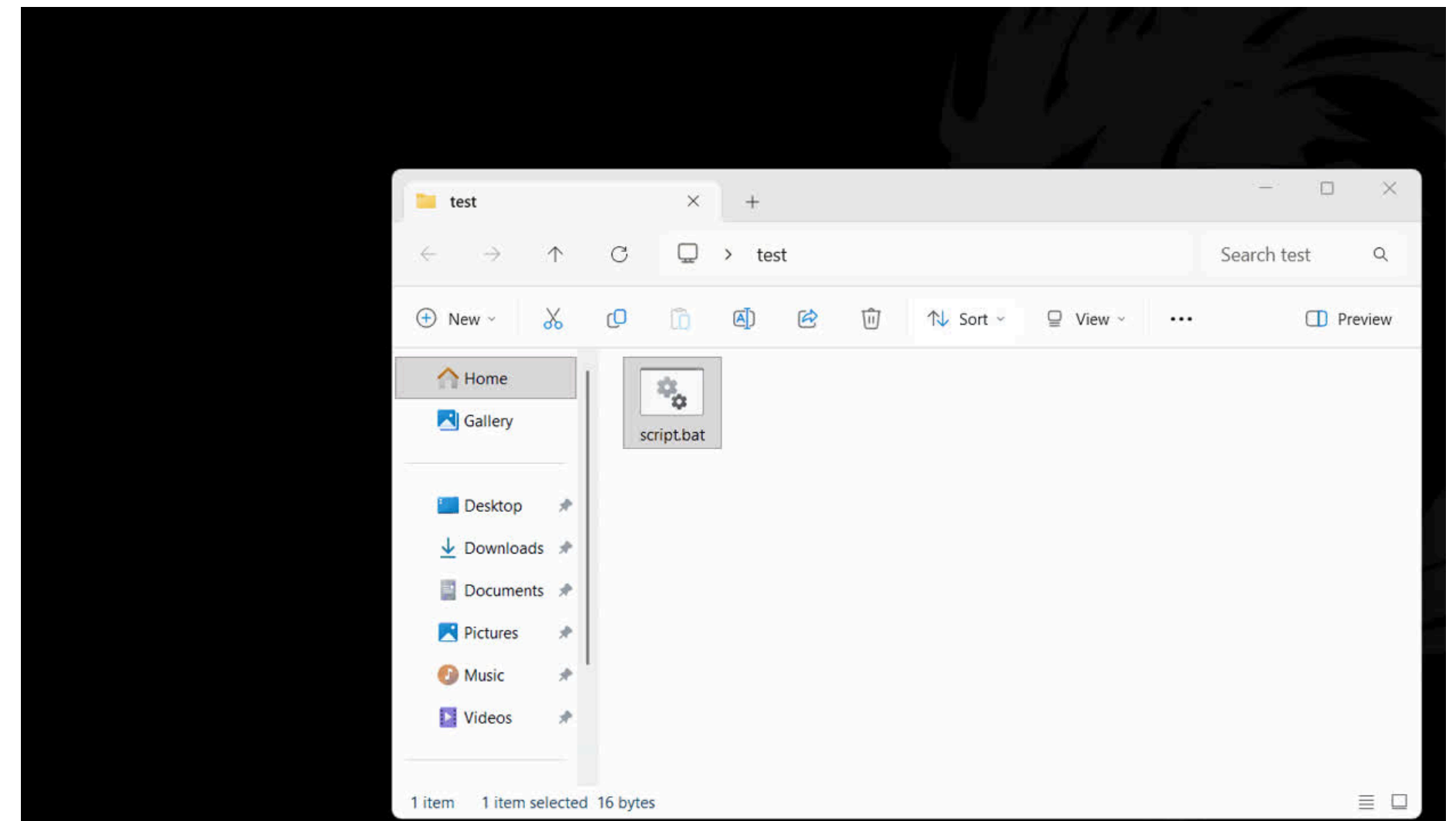
This is the easiest way to run a **.bat file**. It works just like opening any other file on your computer.

Locate the batch file you created.



What Happen:

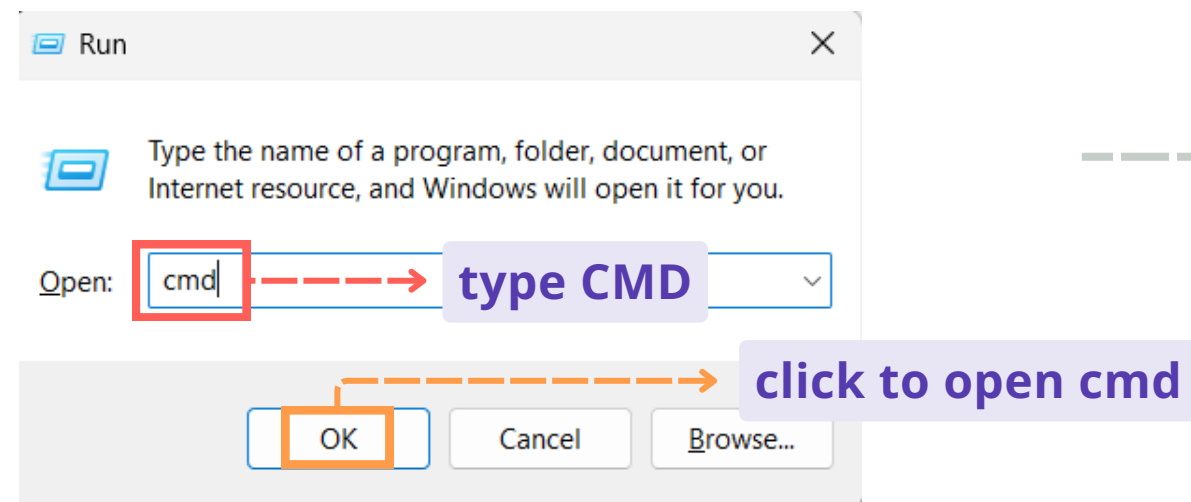
- Command Prompt opens,
- runs the commands,
- then closes when finished.



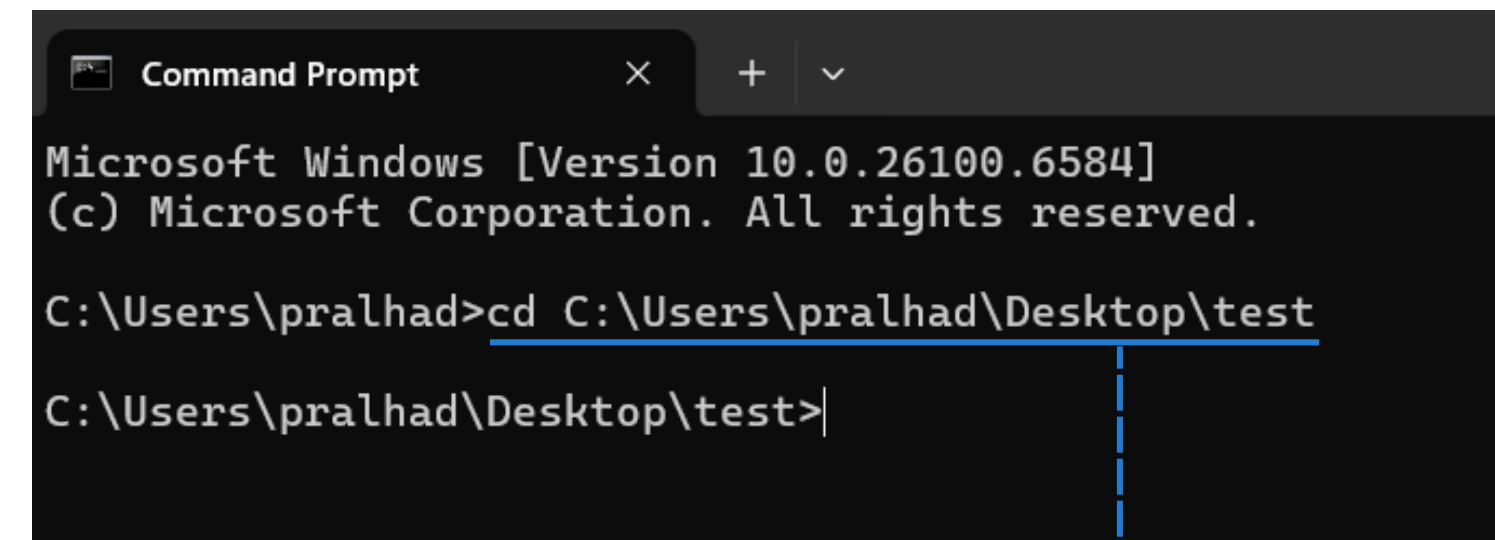
Method 2: Run Using the Command Prompt

1. Open the Command Prompt

- Press **Win + R** on your keyboard.
- In the small “**Run**” dialog box that appears, **type**:

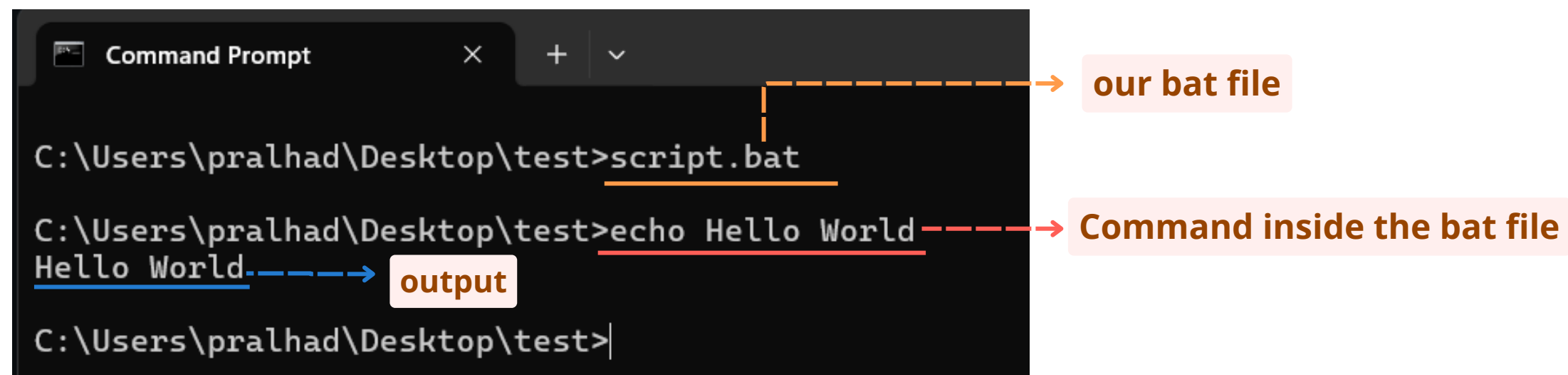


2. Navigate to the folder



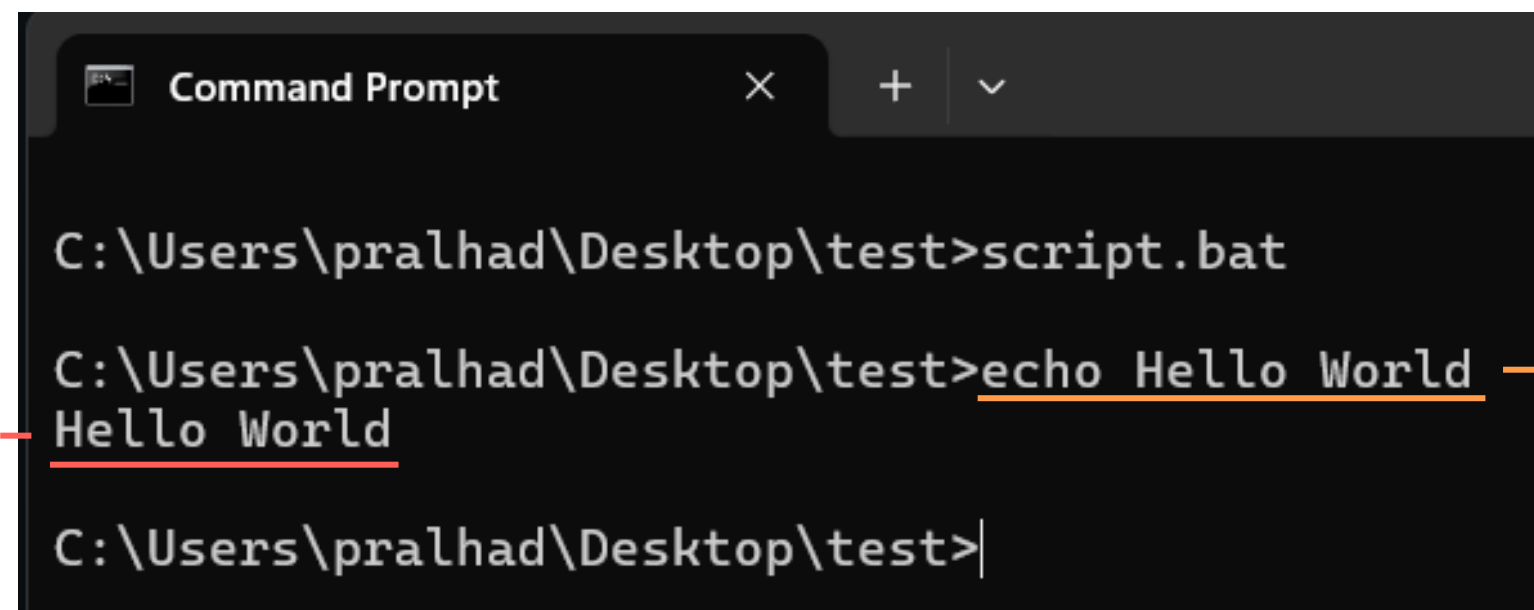
3: Run the batch file

Type the **name of the file**, including the **.bat extension**. For example:



Why Do We See the Command and the Output Separately in the Terminal?

- When we run a **.bat** file from **Command Prompt**, Windows processes the batch file **line by line**.
- For each command, the terminal:



```
C:\Users\pralhad\Desktop\test>script.bat
C:\Users\pralhad\Desktop\test>echo Hello World
Hello World
C:\Users\pralhad\Desktop\test>|
```

Show the command itself

Output of given command

Why Use These Methods?

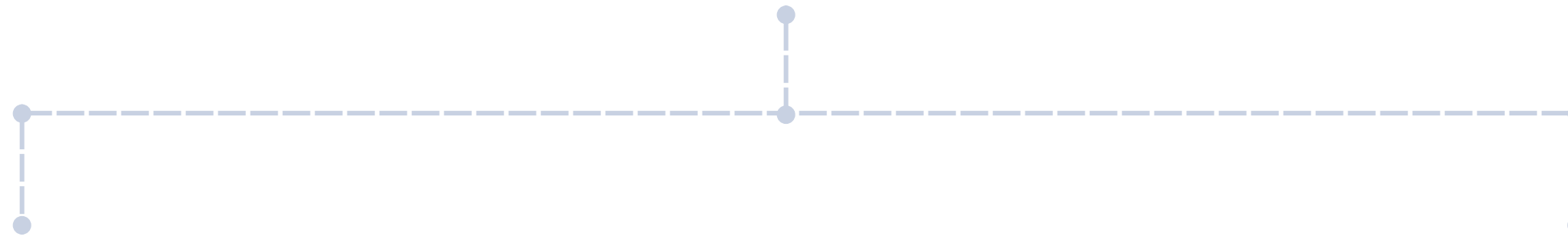
◆ Method 1: Double-Clicking the .bat File

- It's **simple** and **fast**.
- You **don't** need to open **Command Prompt manually** or type **any commands**.
- As long as the file is set up **correctly**, this method is ideal for **quickly** running your script.

◆ Method 2: Running the .bat File via Command Prompt

- It allows you to **see the full output** of the **script**.
- If there are any **errors**, they will **appear** in the **same window**.
- This method is **useful** for **debugging** or when you want to understand what's happening **step by step**.

So far, we've looked at two ways to run a **.bat (batch)** file:



By double-clicking the file

- The script runs in a new **CMD Window**
- but the window **closes** immediately.

By running it manually in the Command Prompt

We see both:

```
Command Prompt
C:\Users\pralhad\Desktop\test>script.bat
C:\Users\pralhad\Desktop\test>echo Hello World
Hello World
C:\Users\pralhad\Desktop\test>
```

Output

Command being executed

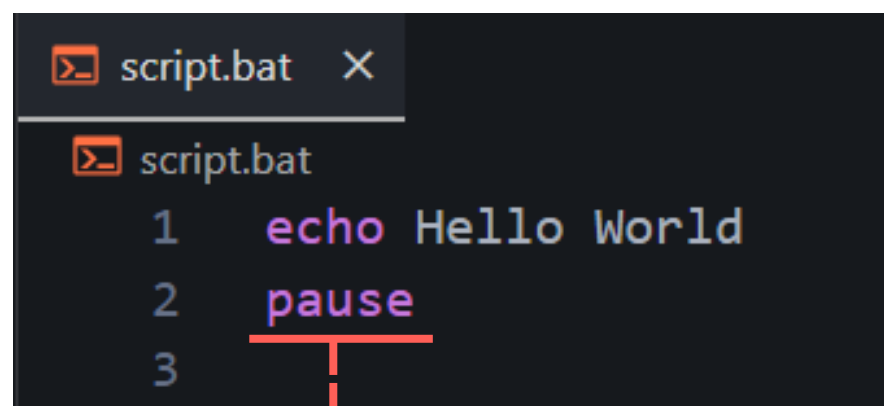
Why Does the Window Close Immediately?

- When we **double-click** a **.bat** file,
 - Windows opens **Command Prompt**,
 - runs the **commands**, and
 - **closes the window as soon as the last command finishes.**
- This is **normal behavior** in Windows — the terminal is only open while it has work to do.
- **Result:** if our script prints something, it flashes too quickly to read.

How to Keep the Terminal Open?

- By default, when a **batch file finishes**, the terminal window **closes immediately**.
- To prevent this, add the **pause** command at the end of batch file.

Update Bat File

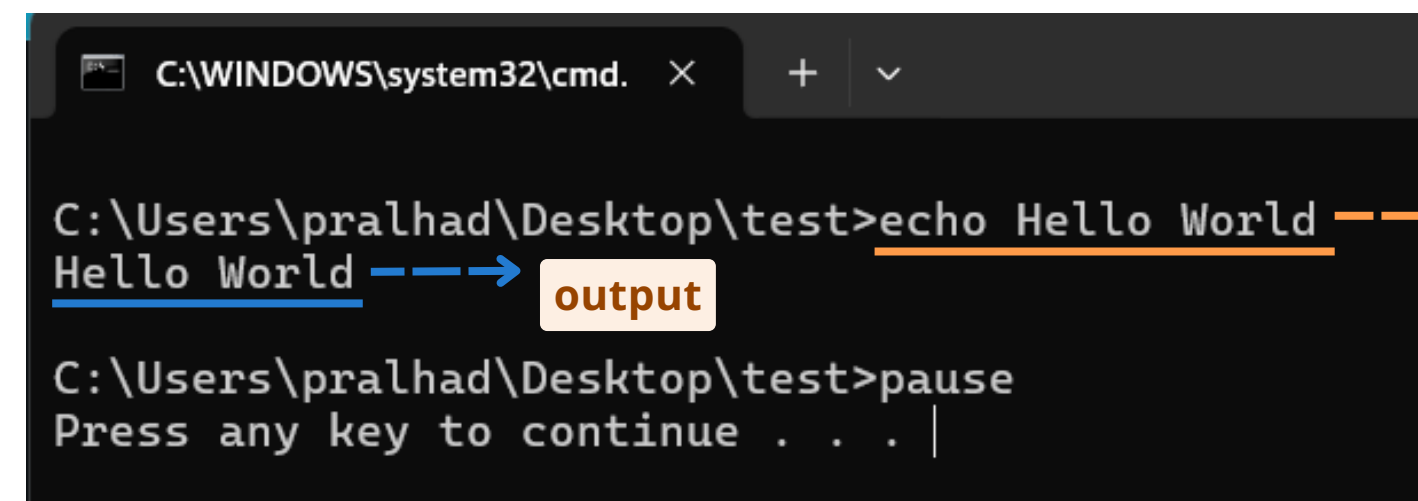


```
script.bat
1 echo Hello World
2 pause
3
```

Keeps the window open

- *waits for a key press before closing.*

Run File



```
C:\WINDOWS\system32\cmd.
C:\Users\pralhad\Desktop\test>echo Hello World
Hello World
C:\Users\pralhad\Desktop\test>pause
Press any key to continue . . .
```

Command being executed

- *The message appears.*
- *The window stays open until you press a key.*

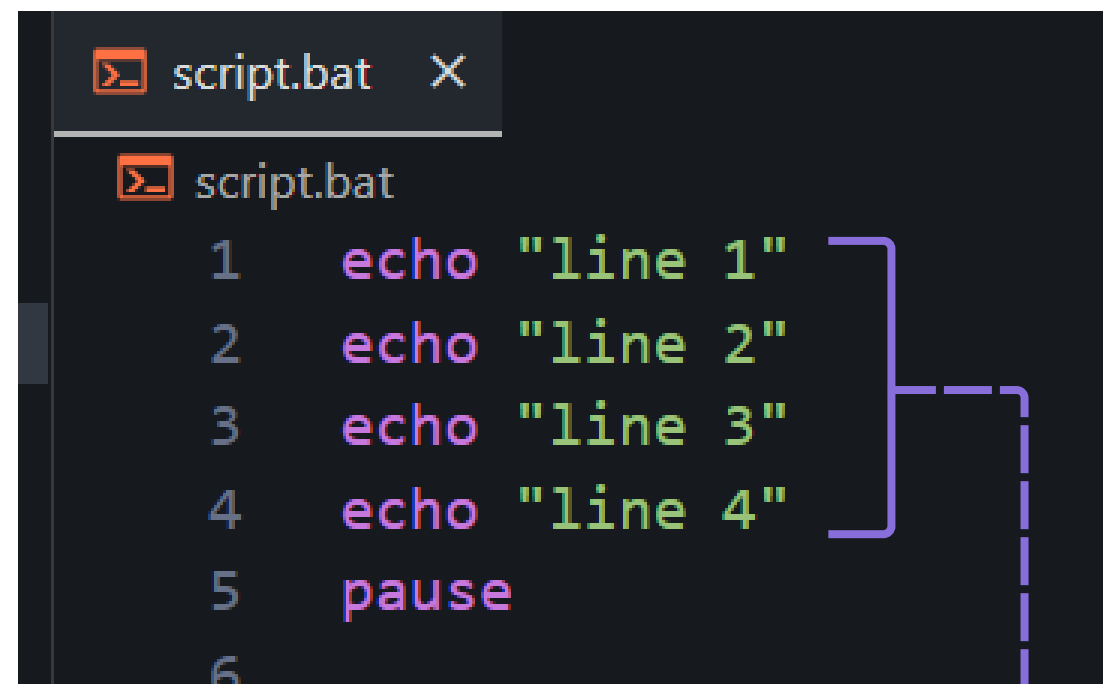
Why Use pause?

- Lets you read output before the window closes.
- Useful for debugging or learning how your batch file runs.
- In production, it's usually removed so the script runs and closes without waiting.

Writing Multiple Commands in a Batch File

- You can add more than **one command** to a batch file.
- This is useful for **automating a sequence of tasks**.

Update a Batch File with Multiple Commands

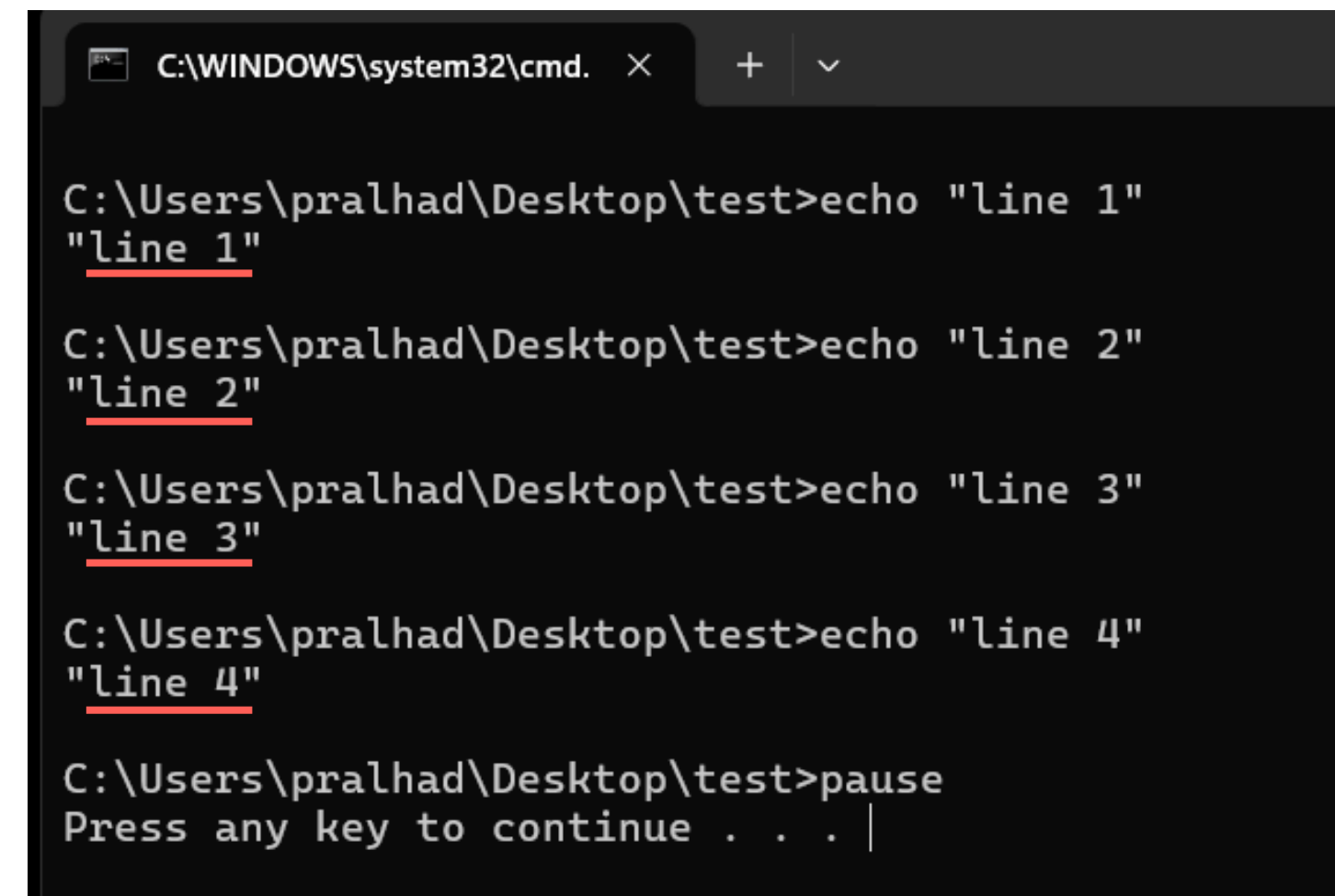


```
script.bat X
script.bat
1  echo "line 1"
2  echo "line 2"
3  echo "line 3"
4  echo "line 4"
5  pause
6
```

Multiple Prints



Output When You Run the File



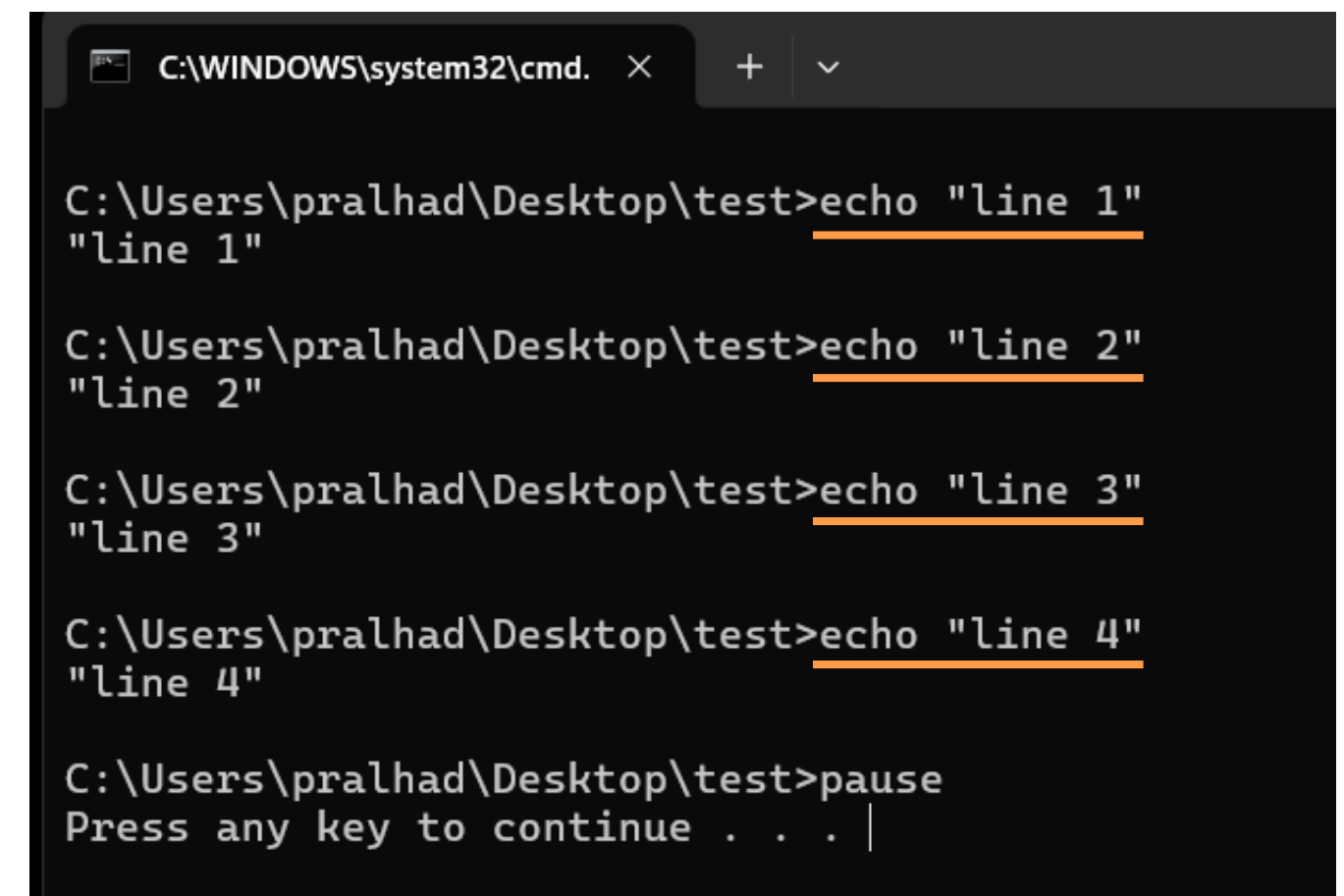
```
C:\WINDOWS\system32\cmd. X + v
C:\Users\pralhad\Desktop\test>echo "line 1"
"line 1"
C:\Users\pralhad\Desktop\test>echo "line 2"
"line 2"
C:\Users\pralhad\Desktop\test>echo "line 3"
"line 3"
C:\Users\pralhad\Desktop\test>echo "line 4"
"line 4"
C:\Users\pralhad\Desktop\test>pause
Press any key to continue . . . |
```


What's Happening?

- Each line in the batch file runs one after the other.
- echo prints the message.
- pause keeps the window open until you press a key.

Why Do You See Extra Text?

- The echo command prints messages in the terminal.
- By default, Windows also shows the command being executed.
 - **Example: C:\...>echo "line 1"**
- This can clutter the output if you only want to see the results.



```
C:\WINDOWS\system32\cmd.  X  +  v

C:\Users\pralhad\Desktop\test>echo "line 1"
"line 1"

C:\Users\pralhad\Desktop\test>echo "line 2"
"line 2"

C:\Users\pralhad\Desktop\test>echo "line 3"
"line 3"

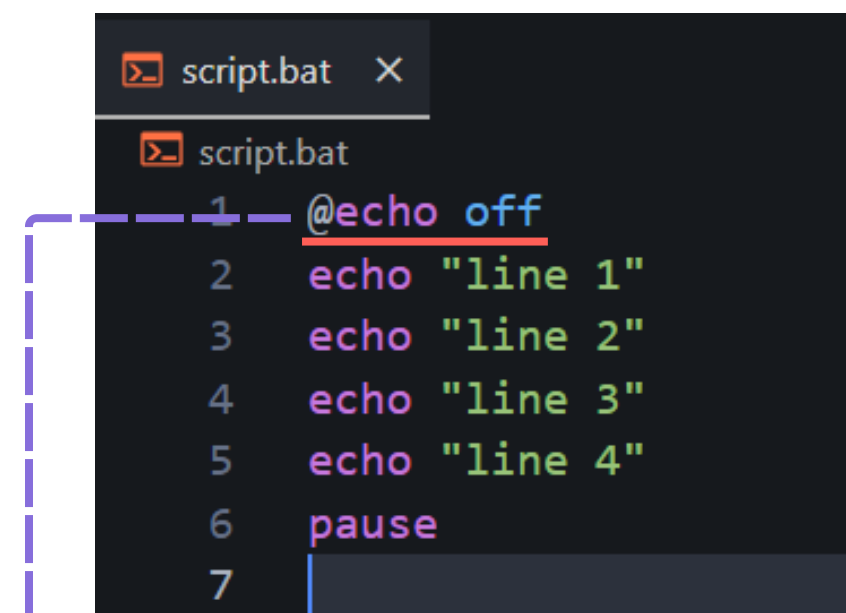
C:\Users\pralhad\Desktop\test>echo "line 4"
"line 4"

C:\Users\pralhad\Desktop\test>pause
Press any key to continue . . . |
```

Clean Up the Output with **@echo off**

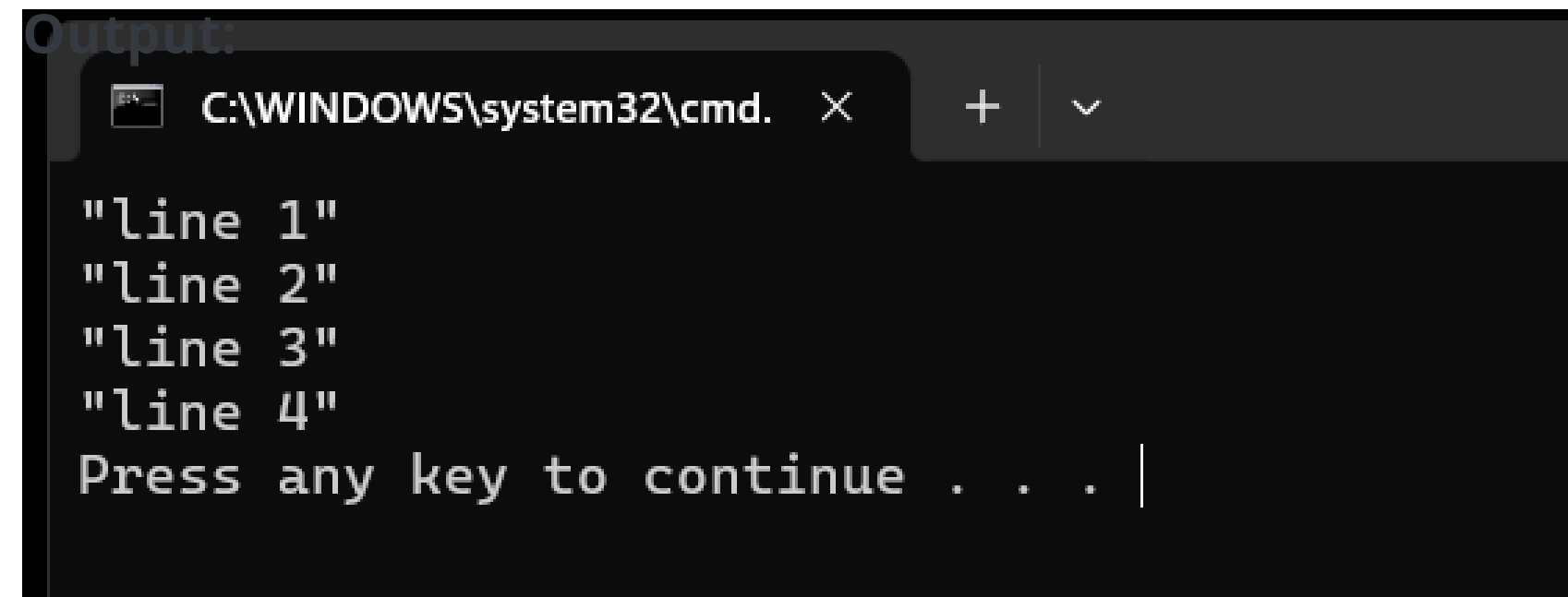
Update the Batch File to Use @echo off:

Add **@echo off** at the top of our file:



```
script.bat x
script.bat
1 @echo off
2 echo "line 1"
3 echo "line 2"
4 echo "line 3"
5 echo "line 4"
6 pause
7
```

Hides the command in output



Output:

```
C:\WINDOWS\system32\cmd. x + v
"line 1"
"line 2"
"line 3"
"line 4"
Press any key to continue . . . |
```

- **@echo off** → **hides** the commands themselves, showing **only** results.
- The @ ensures the **echo off** line itself doesn't get printed.

Using the timeout Command

- Sometimes you don't want the **terminal** to wait for you to **press a key**
- We just want it to stay open for a few seconds and then **close automatically**.
- That's where **timeout** is useful.

Replace pause with timeout

```
script.bat X
script.bat
1 @echo off
2 echo "line 1"
3 echo "line 2"
4 echo "line 3"
5 echo "line 4"
6 timeout 3
7
```

tells the terminal to wait 3 seconds before closing.

It Waits 3 seconds then **CMD** close Automatically

```
C:\WINDOWS\system32\cmd. X + v
"line 1"
"line 2"
"line 3"
"line 4"
Waiting for 2 seconds, press a key to continue ...
```

These Number Updates as per seconds we give in timeout

Why Use timeout Instead of pause ?

- **pause** requires you to press a key,
- while **timeout** waits for a fixed time before continuing.
- It's useful when you want to introduce a **delay** without **manual intervention**.

When to Use

- Use **pause** if you want to **manually check** the output and decide when to close the window.
- Use **timeout** if you want the script to **wait briefly** and then move on or close **without user input**.

Getting Started Commands in .bat Files

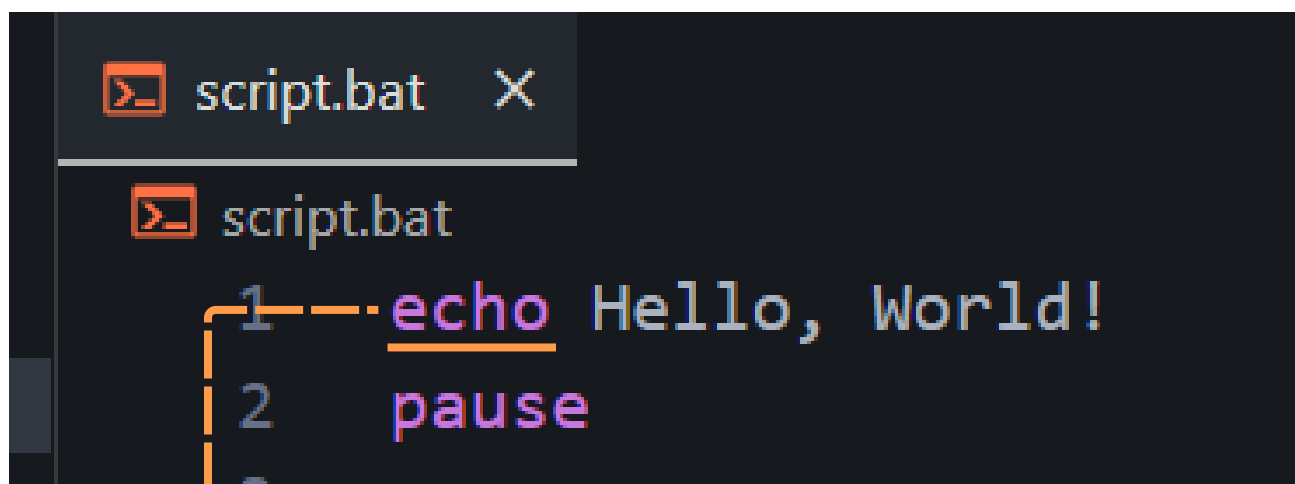
- echo
- pause
- Comments
- Basic Syntax of a .bat File

echo

The echo command has **four** main uses:

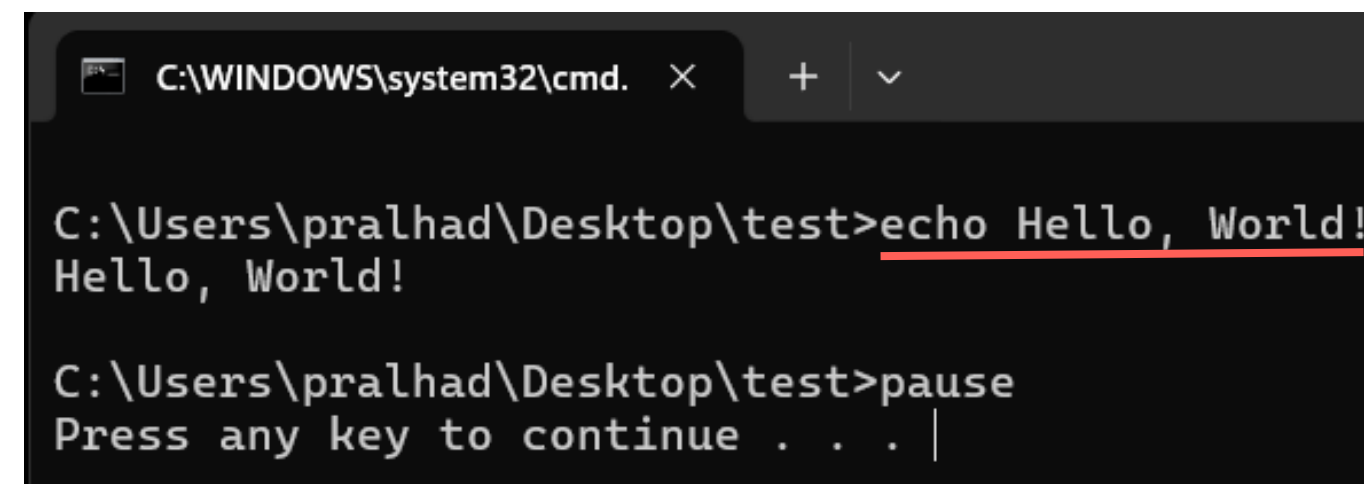
- **Normal echo** → print a message.
- **echo off** → hide command lines (but this command itself is shown once).
- **echo on** → show every command as it runs.
- **@echo off** → hide command lines completely (including the echo off line).

Normal echo (printing a message)



```
script.bat X
script.bat
1 echo Hello, World!
2 pause
```

print a message.



```
C:\WINDOWS\system32\cmd. X + v
C:\Users\pralhad\Desktop\test>echo Hello, World!
Hello, World!
C:\Users\pralhad\Desktop\test>pause
Press any key to continue . . . |
```

echo off

```
script.bat x
script.bat
1  echo off
2  echo Line 1
3  echo Line 2
4  pause
```

hides commands after this line

```
C:\WINDOWS\system32\cmd. x + v
C:\Users\pralhad\Desktop\test>echo off
Line 1
Line 2
Press any key to continue . . . |
```

The line **echo off** is shown once before it takes effect.

echo on

By default, in batch files, command **echoing is ON** unless you disable it with **@echo off**.

```
script.bat x
script.bat
1  echo on
2  echo Line 1
3  echo Line 2
4  pause
5
```

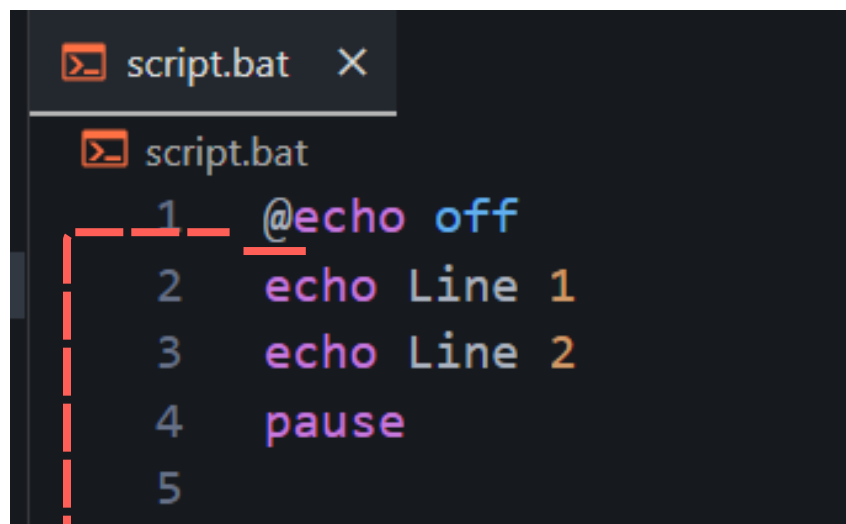
forces showing all commands

```
C:\WINDOWS\system32\cmd. x + v
C:\Users\pralhad\Desktop\test>echo on
C:\Users\pralhad\Desktop\test>echo Line 1
Line 1
C:\Users\pralhad\Desktop\test>echo Line 2
Line 2
C:\Users\pralhad\Desktop\test>pause
Press any key to continue . . . |
```

commands

Here, every command and its output is displayed.

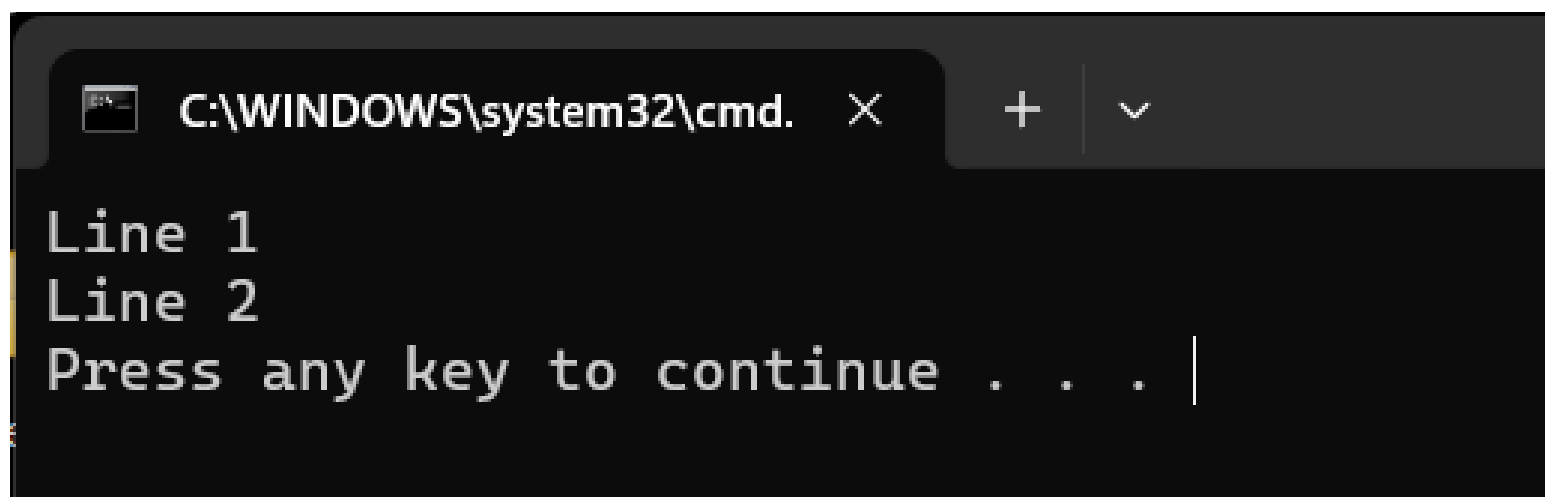
@echo off



```
script.bat
1 @echo off
2 echo Line 1
3 echo Line 2
4 pause
5
```

- The @ symbol tells Windows: **don't show this command either.**
- That's why most **.bat** files begin with **@echo off.**

Output



```
C:\WINDOWS\system32\cmd.
Line 1
Line 2
Press any key to continue . . . |
```


pause

- **Purpose:** Stops the script until the **user presses a key**.
- Commonly used so the terminal **doesn't close before** you **read the output**.

```
script.bat X
script.bat
1 echo Script finished
2 pause
3
```

Keeps the window open

waits for a key press before closing.



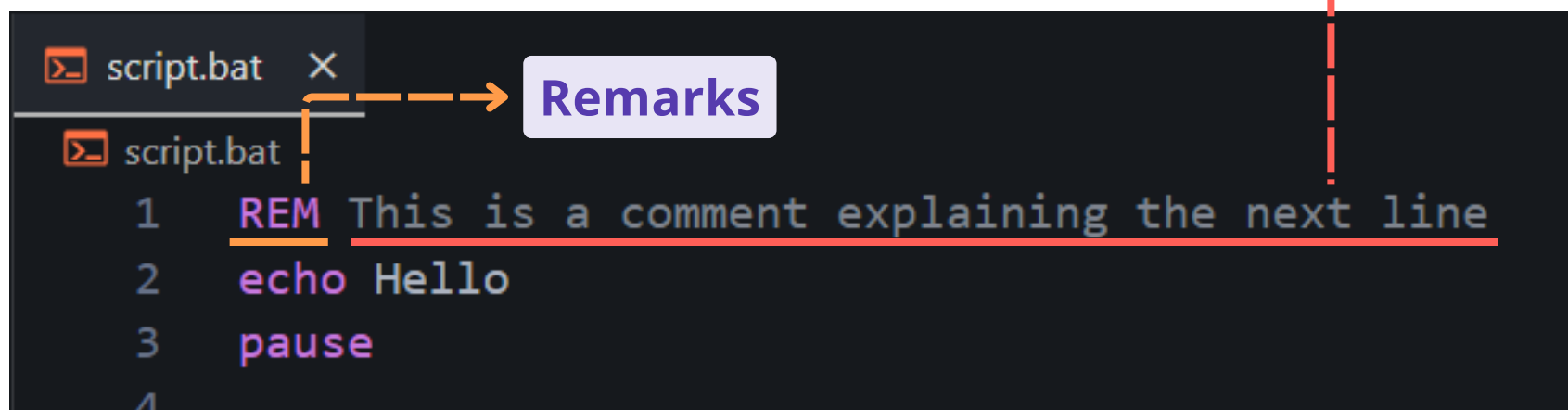
```
C:\WINDOWS\system32\cmd. X + v
C:\Users\pralhad\Desktop\test>echo Script finished
Script finished

C:\Users\pralhad\Desktop\test>pause
Press any key to continue . . . |
```

Comments

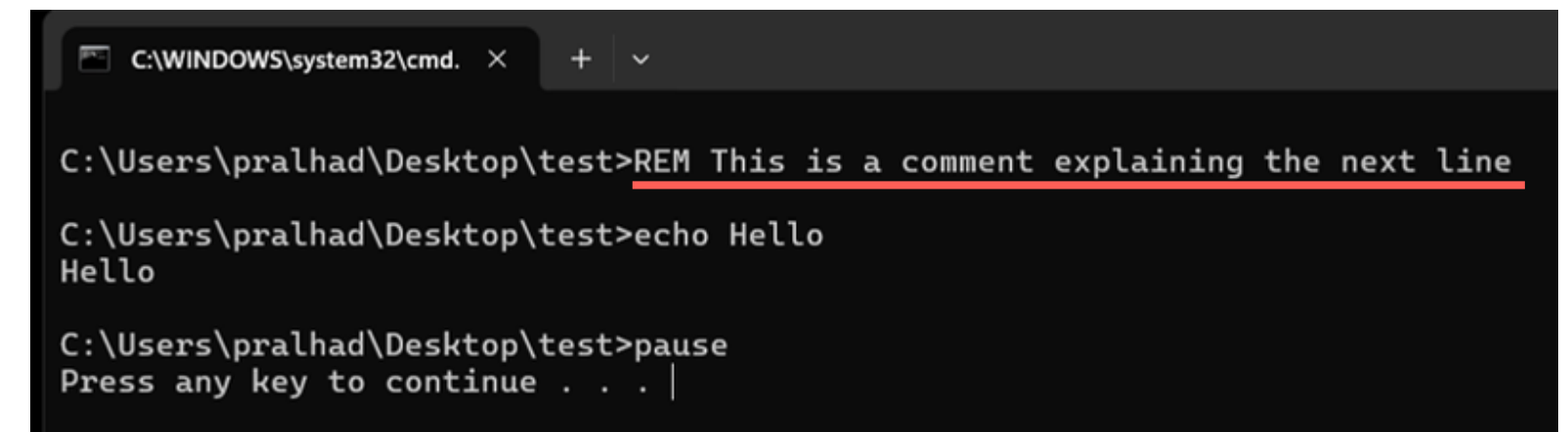
- **Comments** are **notes you add inside a script to explain what the code does**.
- They **do not run** — Windows ignores them.
- This makes scripts easier to understand for yourself and others.

Using REM



A screenshot of a Notepad window titled 'script.bat'. The script contains four lines: line 1 is 'REM This is a comment explaining the next line', line 2 is 'echo Hello', line 3 is 'pause', and line 4 is empty. An orange arrow points from a purple box labeled 'Remarks' to the 'REM' keyword. A red dashed arrow points from a purple box labeled 'Comments' to the entire first line.

```
script.bat
1 REM This is a comment explaining the next line
2 echo Hello
3 pause
4
```



A screenshot of a Windows Command Prompt window titled 'C:\WINDOWS\system32\cmd.'. It shows the execution of a batch script. The first command is 'REM This is a comment explaining the next line', which is underlined in red. The second command is 'echo Hello', which outputs 'Hello'. The third command is 'pause', which outputs 'Press any key to continue . . . |'.

```
C:\WINDOWS\system32\cmd.
C:\Users\pralhad\Desktop\test>REM This is a comment explaining the next line
C:\Users\pralhad\Desktop\test>echo Hello
Hello
C:\Users\pralhad\Desktop\test>pause
Press any key to continue . . . |
```

- **REM** is the traditional way to write comments.
- Good for longer explanations.

How to Hide REM Output

- If you don't want **REM** to appear → Add **@echo off** at the top of the **batch file**.

Using double colons :

```
script.bat X
script.bat
1  :: This is another comment style
2  echo Hello
3  pause
```



Output

```
C:\WINDOWS\system32\cmd. X + v
C:\Users\pralhad\Desktop\test>echo Hello
Hello
C:\Users\pralhad\Desktop\test>pause
Press any key to continue . . . |
```

- Shorter and cleaner than **REM**.
- Often used for **quick inline documentation**.

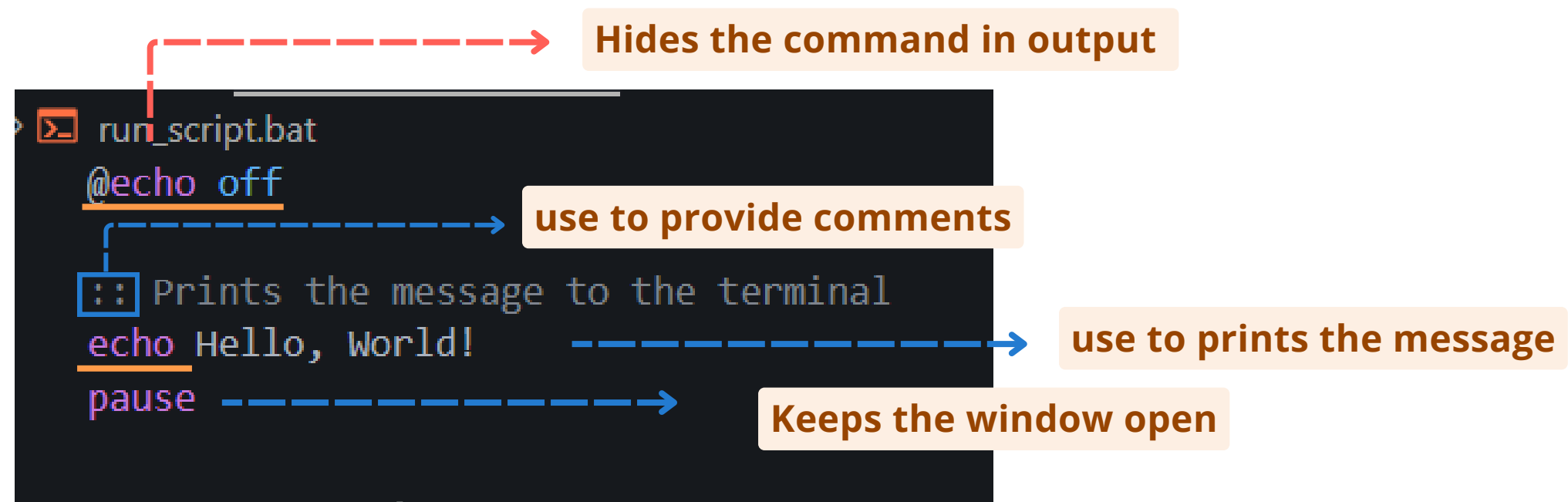
 Key Points

- **REM = official command** → shows up in output unless **echoing is disabled**.
- **:: = ignored pseudo-label** → never shows up, even if **echoing is on**.

Basic Syntax of a .bat File

- A **.bat file** is a plain text file that contains a sequence of commands.
- These commands are executed in **order**, from top to bottom.

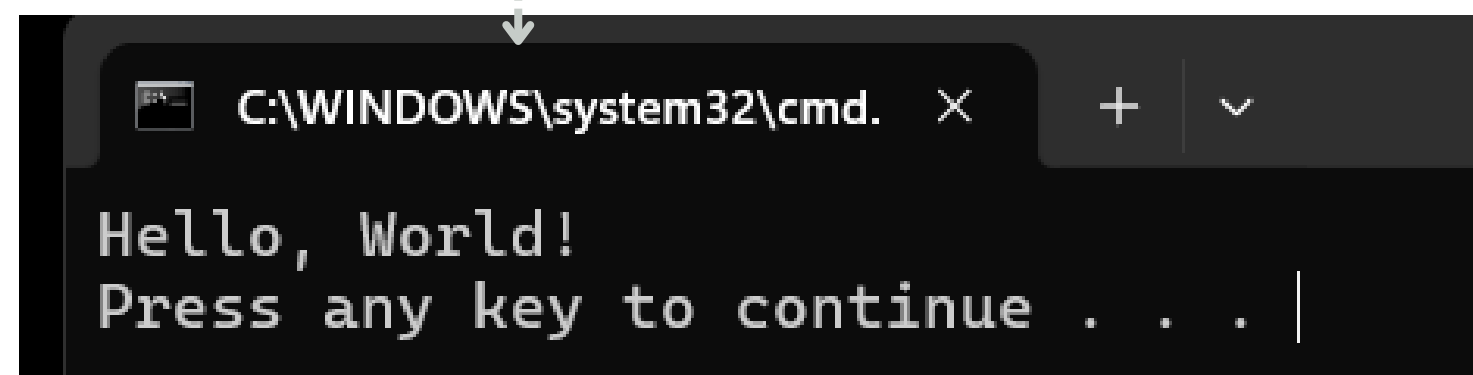
Here's an example of a simple **.bat** file



```
> run_script.bat
@echo off
:: Prints the message to the terminal
echo Hello, World!
pause
```

The diagram shows a code editor window with a .bat file named 'run_script.bat'. The file contains the following commands: '@echo off', ':: Prints the message to the terminal', 'echo Hello, World!', and 'pause'. Annotations with arrows point to each line: a red arrow from '@echo off' points to a box saying 'Hides the command in output'; a blue arrow from ':: Prints the message to the terminal' points to a box saying 'use to provide comments'; a blue arrow from 'echo Hello, World!' points to a box saying 'use to prints the message'; and a blue arrow from 'pause' points to a box saying 'Keeps the window open'.

Output:



```
C:\WINDOWS\system32\cmd.
Hello, World!
Press any key to continue . . . |
```

The screenshot shows a Windows command prompt window titled 'C:\WINDOWS\system32\cmd.'. The output of the .bat file is displayed: 'Hello, World!' followed by 'Press any key to continue . . . |'.

Running Applications with a .bat File

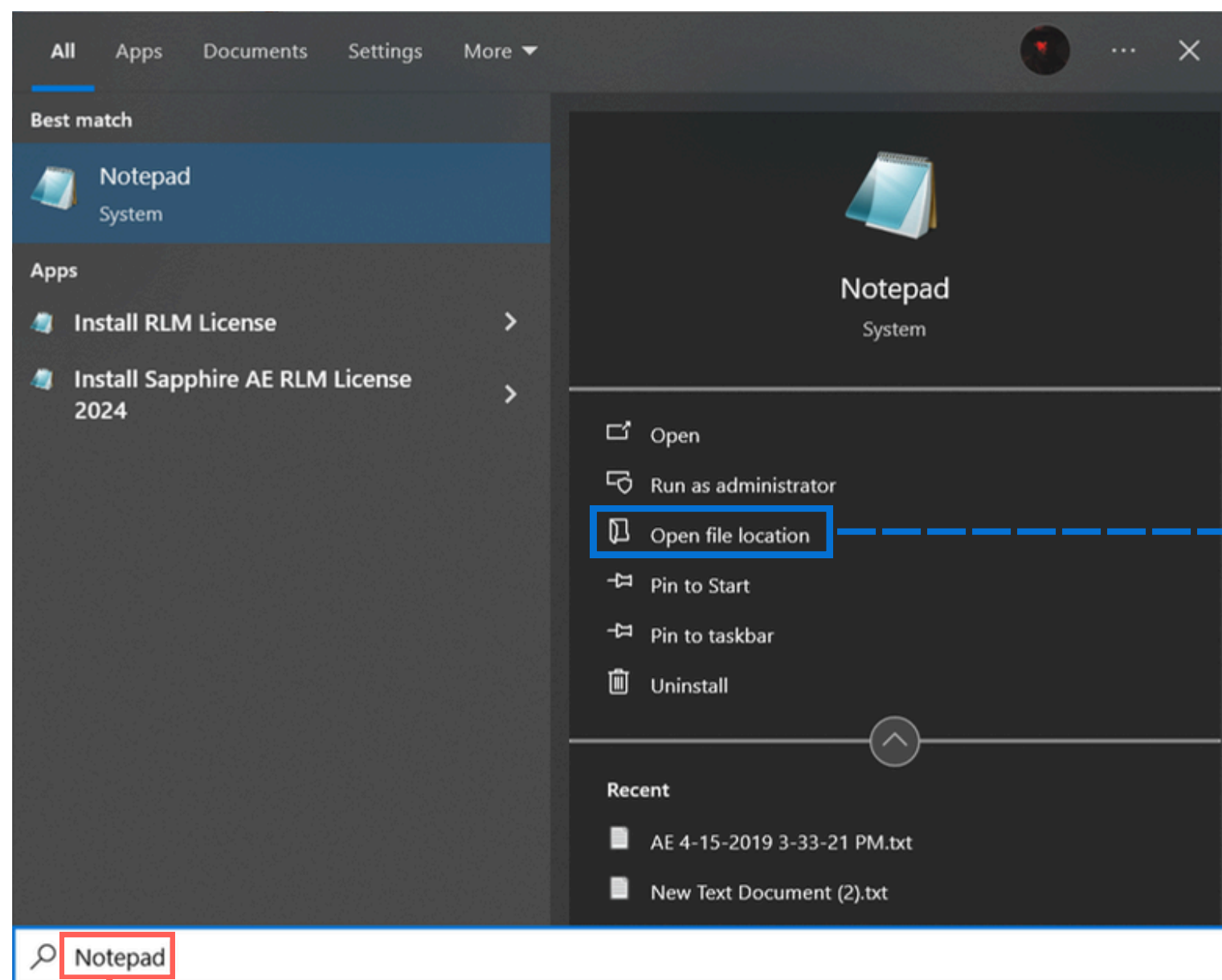
- Launching Applications with Batch Files
- Running Another Application (WordPad)
- Use Case in the VFX Industry
- Studio Scenario
- Common Uses in VFX Work

Launching Applications with Batch Files

Batch files can be used to **open programs** on computer. We'll use **Notepad** as a **simple example**.

Find the Application Path

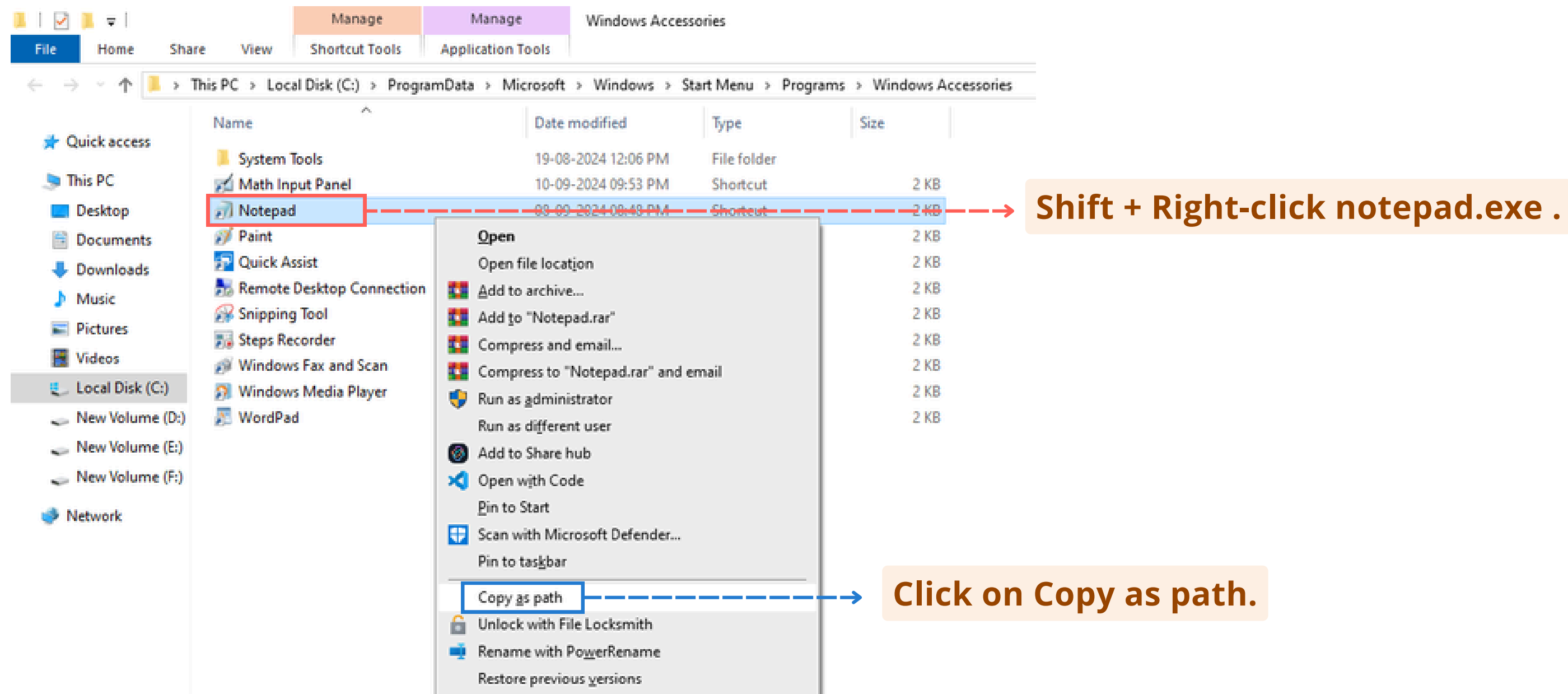
Open the **Start Menu** and search for **Notepad**.



Right-click → Open file location.

Search notepad

In the folder, **right-click the shortcut** → **Open file location** again.

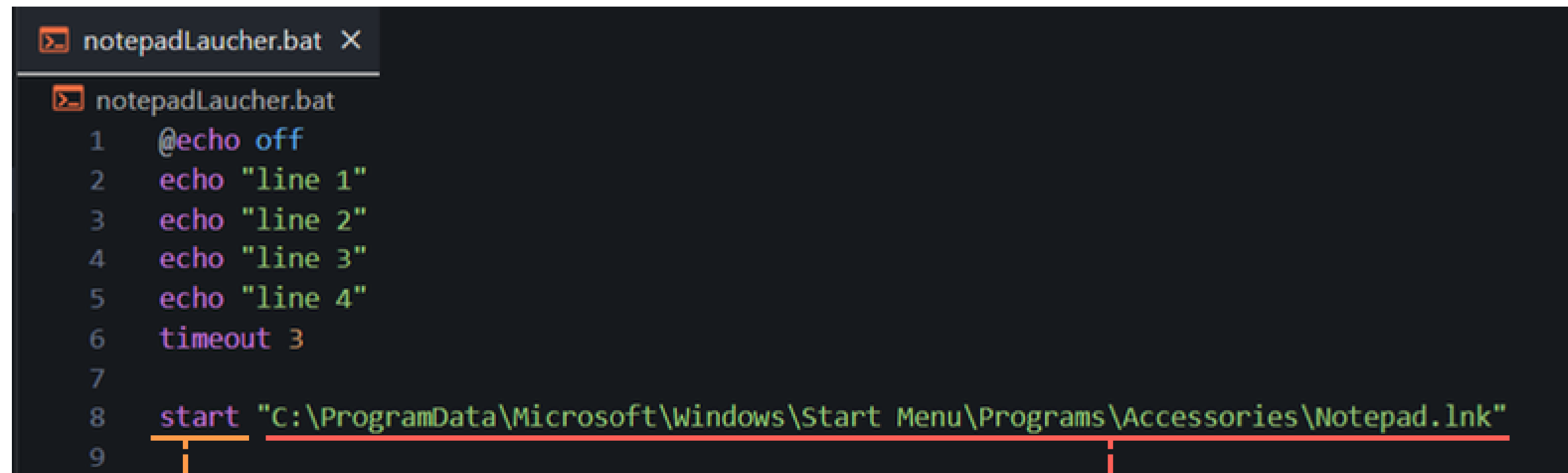


We now have the **path to Notepad**. It's usually something like:

```
"C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Accessories\Notepad.lnk"
```

Create the Batch File

- Now that we have the **path**, let's write a **batch file** to **launch Notepad**.
- **Save** the file as **notepadLauncher.bat**.

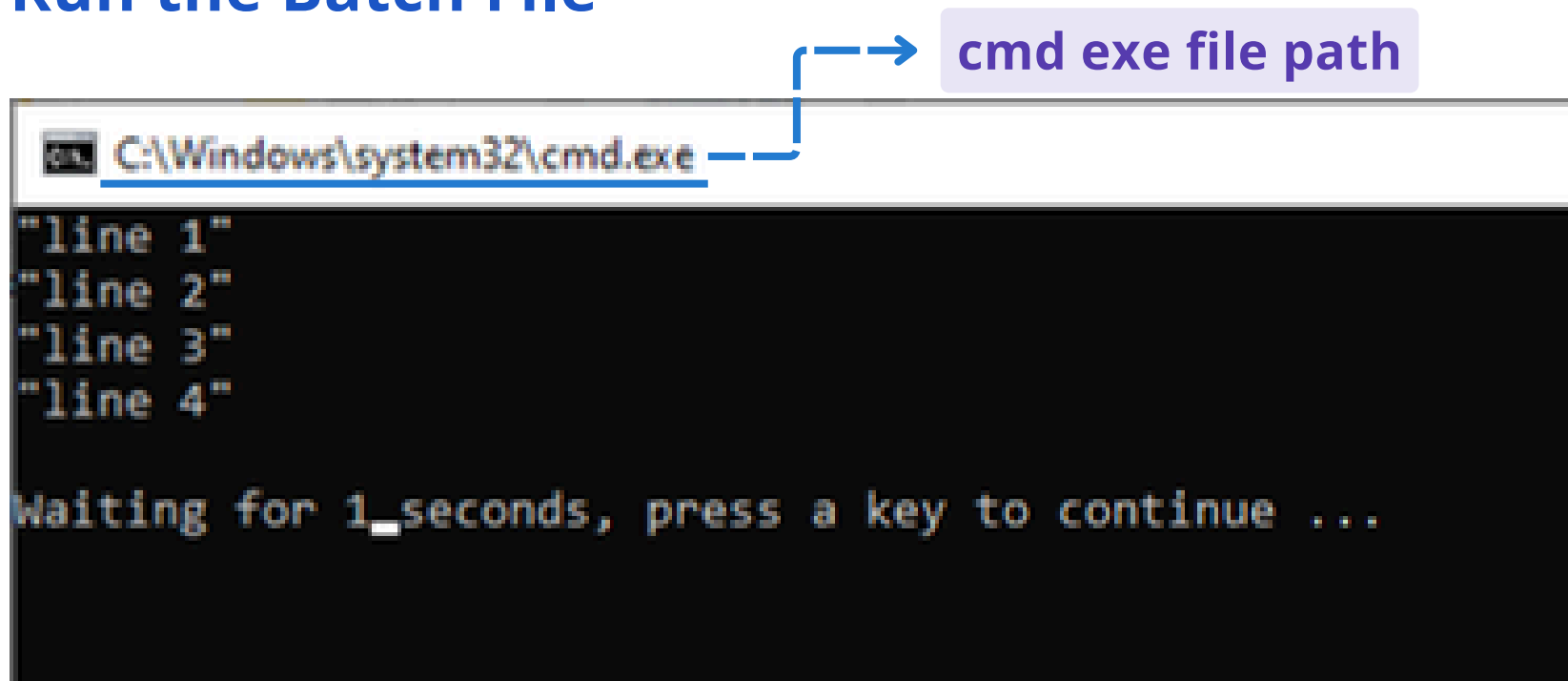


```
notepadLauncher.bat X
notepadLauncher.bat
1 @echo off
2 echo "line 1"
3 echo "line 2"
4 echo "line 3"
5 echo "line 4"
6 timeout 3
7
8 start "C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Accessories\Notepad.lnk"
9
```

→ open a new process or window

→ Absolute path of notepad exe file

Run the Batch File



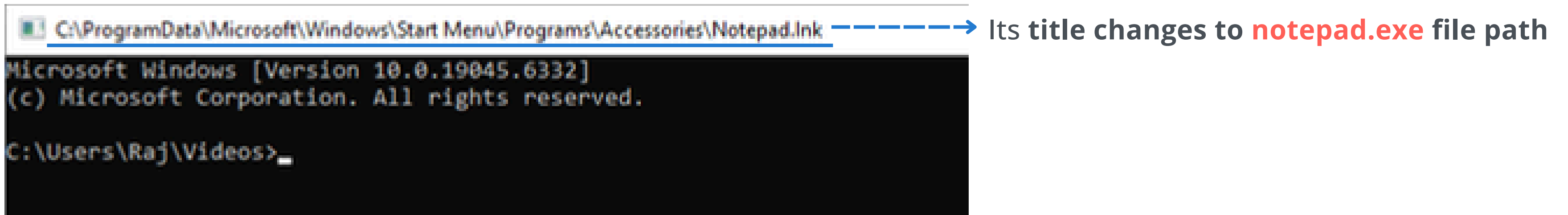
```
C:\Windows\system32\cmd.exe
"line 1"
"line 2"
"line 3"
"line 4"
Waiting for 1_seconds, press a key to continue ...
```

When we double-click **notepadLaucher.bat**:

- The batch file runs and prints the echo lines.
- It waits 3 seconds.

- Then **Notepad opens**, but instead of launching in a separate window

The terminal window stays open.



```
C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Accessories\Notepad.lnk
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Raj\Videos>
```

The Notepad window is launched **inside the terminal's process context**, not separately.

Why Does This Happen?

When we use start like this:

```
7  
8 start "C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Accessories\Notepad.lnk"  
9
```

✗ Not a Program path

Because of that, **Notepad** doesn't launch separately

Fix the Problem

- Use start "" "C:\path\to\app.exe" → the "" is an empty window-title **placeholder**
- **Without** it **Windows can misinterpret a quoted path as the title** and not launch the EXE correctly.

```
start "" "C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Accessories\Notepad.lnk"
```

optional window title for the new window.

The program or file path we want to run.

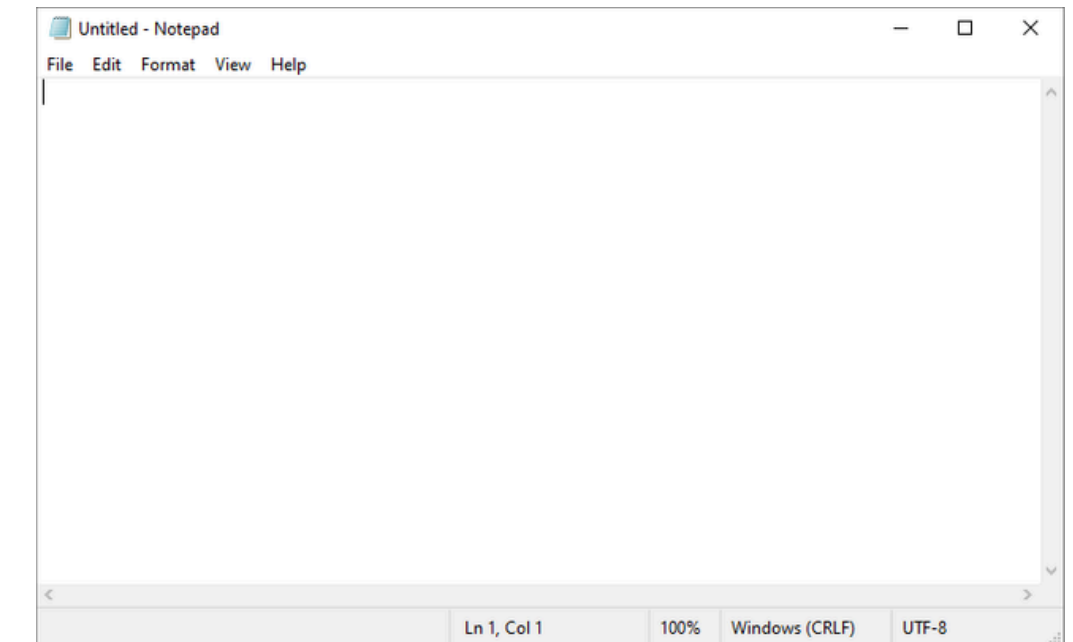
Output

```
start "" "C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Accessories\notepad.exe"
```

optional window title for the new window.

The program or file path we want to run.

Output



Why this is useful in VFX:

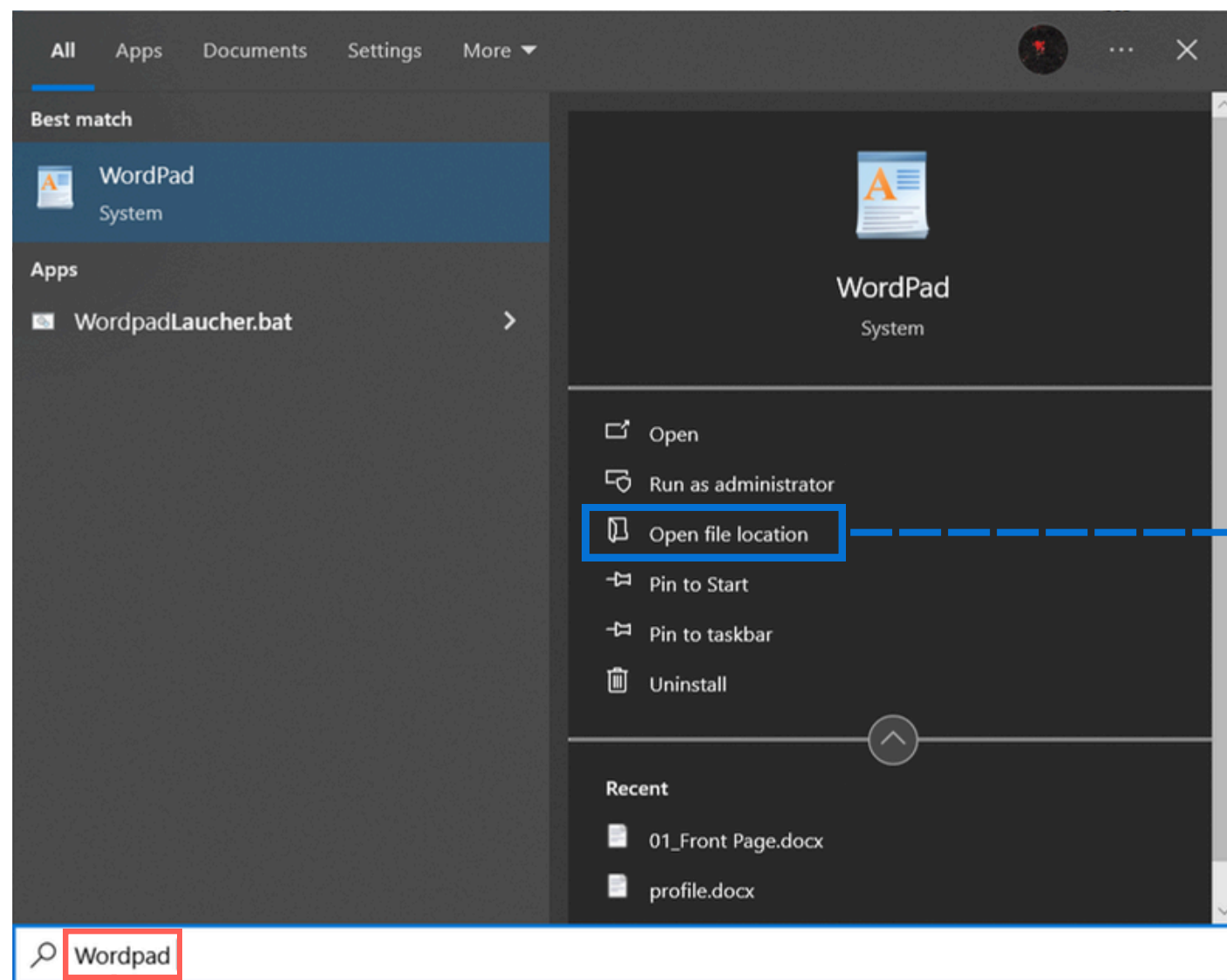
- We can open programs (like Notepad, Maya, Nuke, etc.) using one file.
- We can run custom scripts or tools before starting the main application.
- It saves time by combining multiple steps into one double-click.
- It helps create a repeatable setup for your tools — useful in a production pipeline.

Running Another Application (WordPad)

- Now that we've seen how to launch Notepad **using a batch file**,
- let's apply the same process to another application — **WordPad**.

Find the Path to WordPad

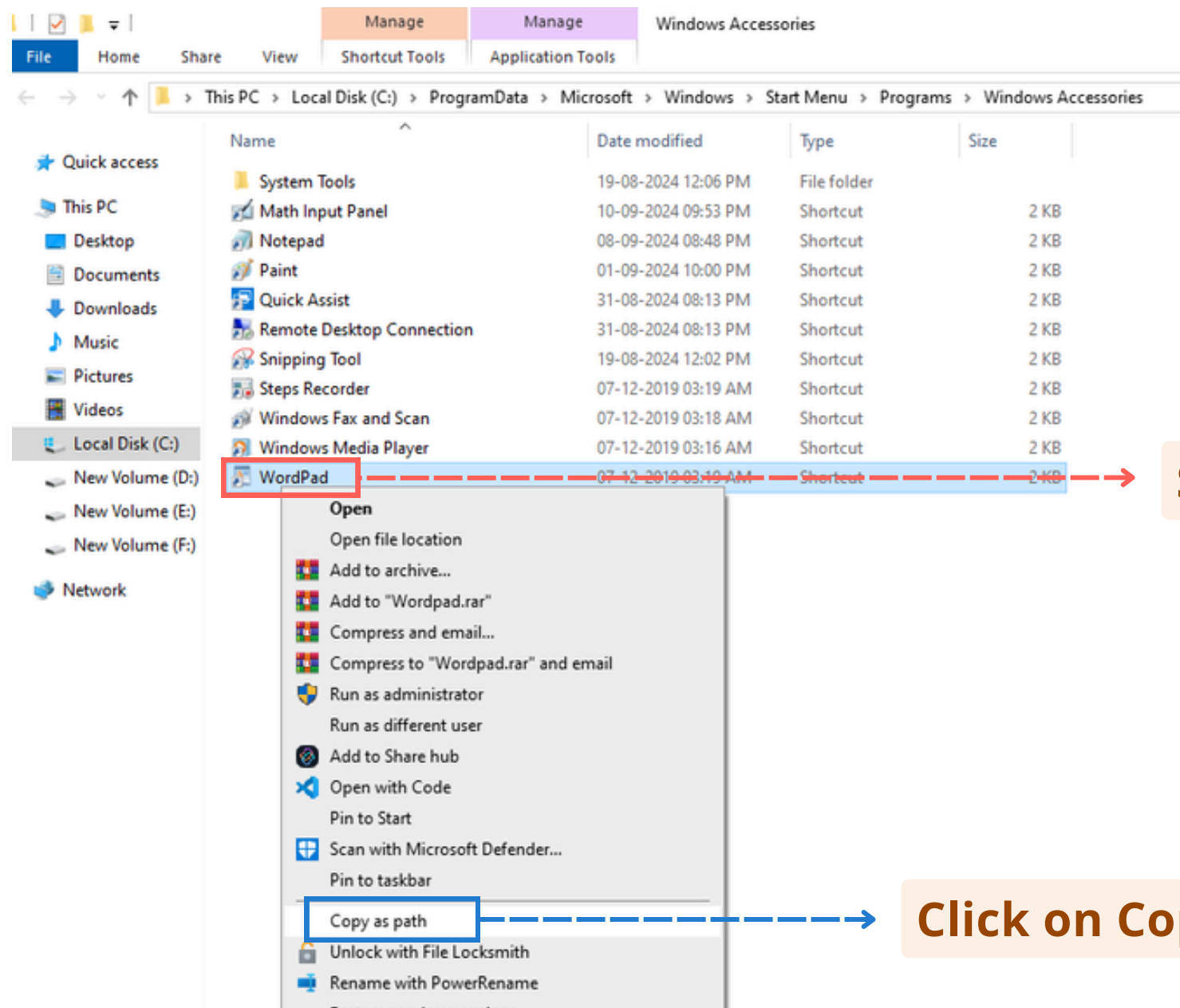
Search for WordPad in the **Start Menu**



Right-click → Open file location.

Search Wordpad

In the folder, **right-click the shortcut** → **Open file location** again.



Shift + Right-click wordpad.exe .

Click on Copy as path.

We now have the **path to Notepad**. It's usually something like:

```
"C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Accessories\Wordpad.lnk"
```


Modify the Batch File

Update your batch file to launch **WordPad** instead of **Notepad**. we use **WordpadLaucher.bat**

```
WordpadLaucher.bat X
WordpadLaucher.bat
1 @echo off
2 echo "line 1"
3 echo "line 2"
4 echo "line 3"
5 echo "line 4"
6 timeout 3
7
8 start "" "C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Accessories\Wordpad.lnk"
9
```

Diagram labels:

- Window Title (points to the empty string "" in line 8)
- Path of exe file (points to the file path in line 8)

Notes

- The "" is required.
 - It acts as a placeholder for the window title.
 - Without it, Windows may mistake the file path for a title, and the program may not launch properly.

Save & Run File

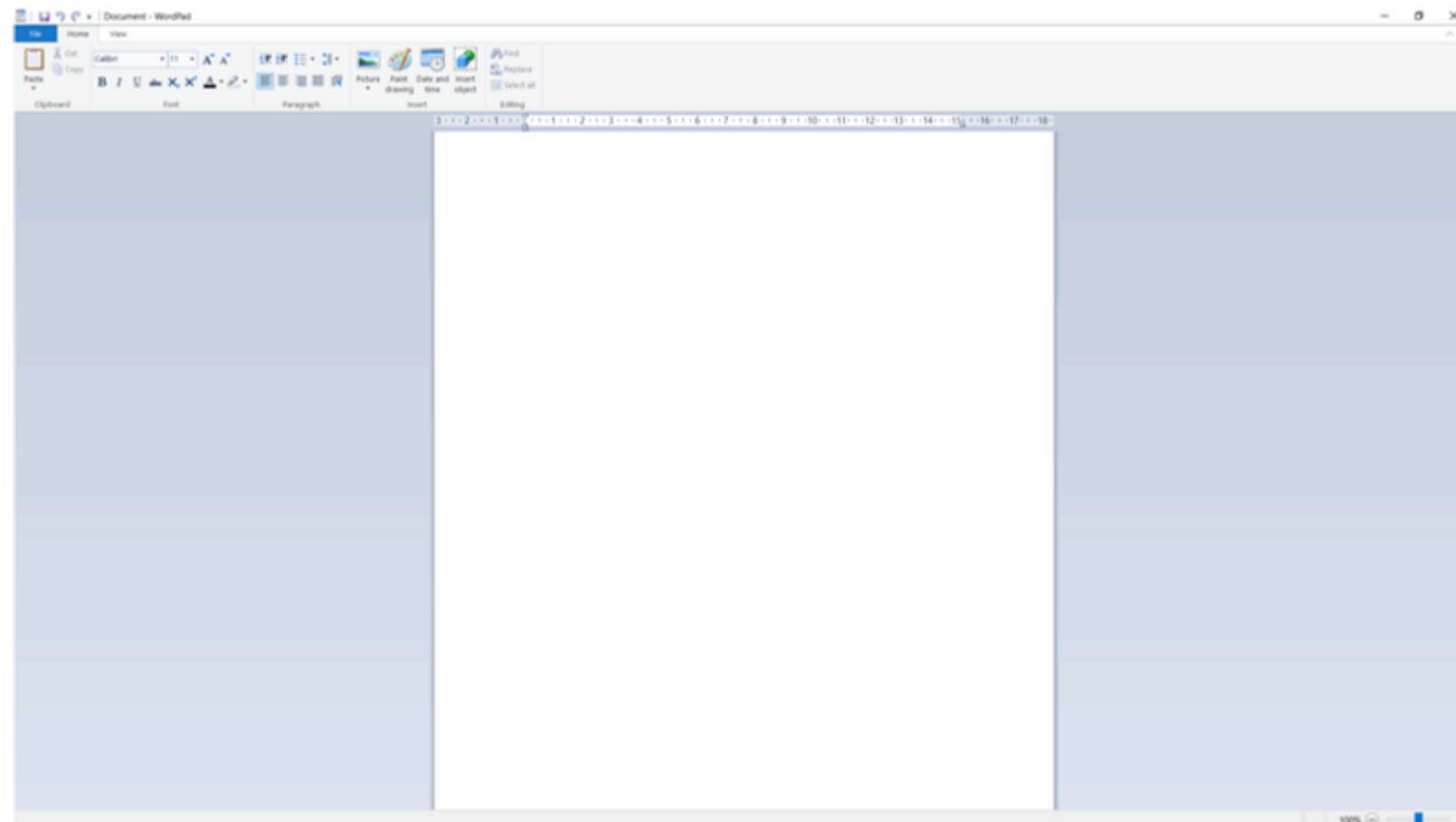
C:\Windows\system32\cmd.exe

```
"line 1"  
"line 2"  
"line 3"  
"line 4"
```

print echo lines

Waiting for 2 seconds, press a key to continue ...

Waits 3 sec → because of timeout 3

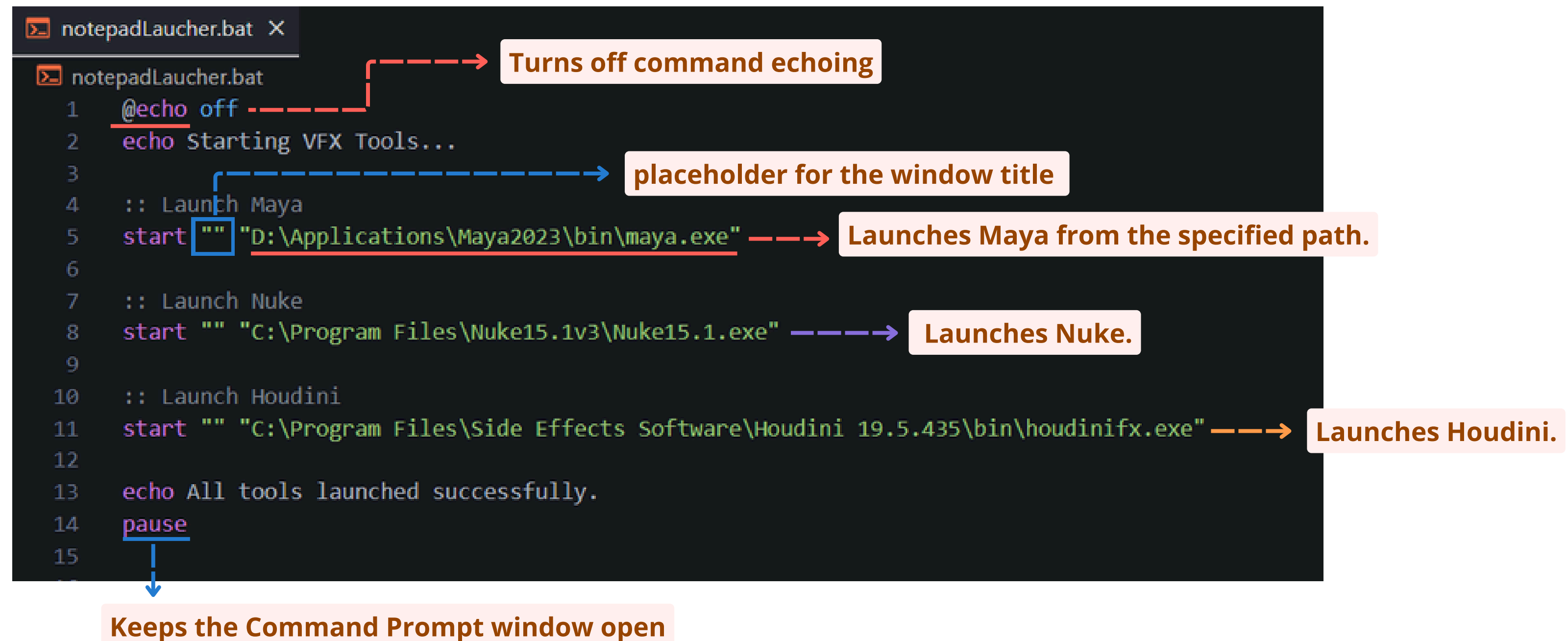


WordPad launches in its own separate window, as expected.

Use Case in the VFX Industry

Launch DCC Applications Simultaneously

Objective: Start Maya, Nuke, and Houdini with a single double-click.



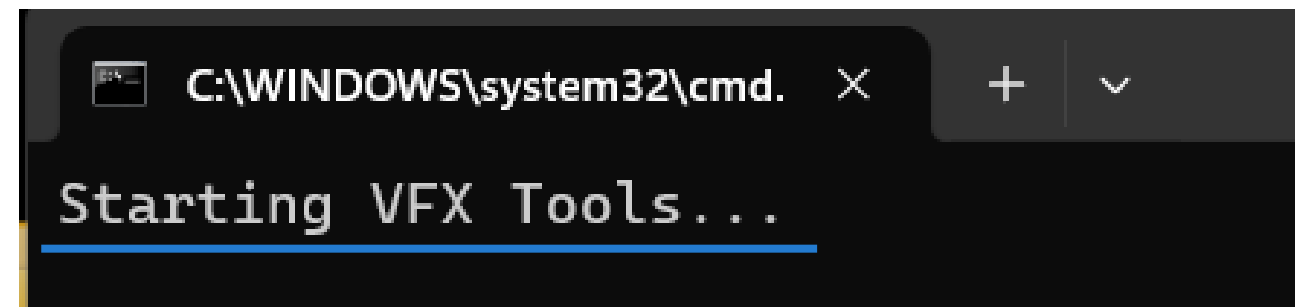
```
notepadLaucher.bat X
notepadLaucher.bat
1  @echo off
2  echo Starting VFX Tools...
3
4  :: Launch Maya
5  start "" "D:\Applications\Maya2023\bin\maya.exe"
6
7  :: Launch Nuke
8  start "" "C:\Program Files\Nuke15.1v3\Nuke15.1.exe"
9
10 :: Launch Houdini
11 start "" "C:\Program Files\Side Effects Software\Houdini 19.5.435\bin\houdinifx.exe"
12
13 echo All tools launched successfully.
14 pause
15
```

Annotations:

- Turns off command echoing (points to line 1: `@echo off`)
- placeholder for the window title (points to line 3: empty string in `start ""`)
- Launches Maya from the specified path. (points to line 5: `"D:\Applications\Maya2023\bin\maya.exe"`)
- Launches Nuke. (points to line 8: `"C:\Program Files\Nuke15.1v3\Nuke15.1.exe"`)
- Launches Houdini. (points to line 11: `"C:\Program Files\Side Effects Software\Houdini 19.5.435\bin\houdinifx.exe"`)
- Keeps the Command Prompt window open (points to line 14: `pause`)

What will happen when we run Bat file ?

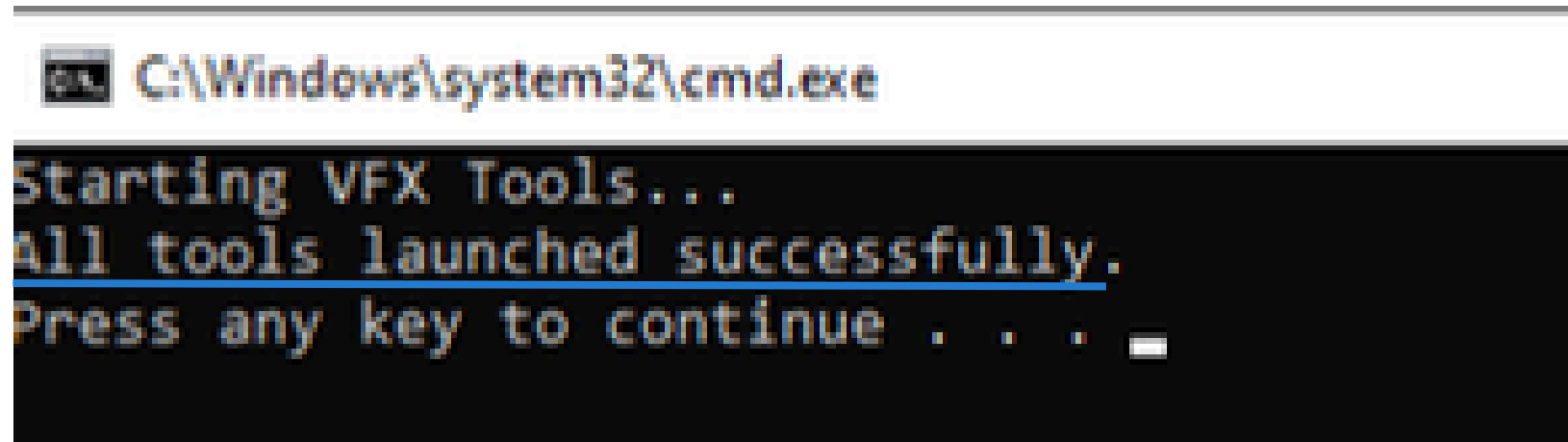
First, a Command Prompt window opens and displays



Then, **Maya**, **Nuke**, and **Houdini** begin launching **one by one** in the background.



Once all tools are triggered, the terminal updates with:



```
C:\Windows\system32\cmd.exe

Starting VFX Tools...
All tools launched successfully.
Press any key to continue . . .
```

The terminal stays open (because of pause), so you can confirm everything launched.

Common uses include:

- Launching multiple tools at once.
- Opening project files in the correct software.
- Setting environment variables before a tool starts.

Studio Scenario

Scenario 1 – Startup Automation

A **VFX artist** arrives in the morning. Instead of opening **Maya**, **Nuke**, and **Houdini** one by one, they:

- Double-click **launch_tools.bat**.
- All three applications open within seconds.
- The batch file confirms success with messages in the **terminal**.

Scenario 2 – Review Preparation

- A **compositor** needs to review the latest renders. Instead of:
 - Browsing folders and copying files
 - Moving assets to the review directory
 - Manually opening Nuke with the right script
- They simply:
 - Double-click **prepare_review.bat**
 - Renders are copied automatically
 - Nuke opens with the correct project

Batch files cut manual prep, so artists can focus on feedback.

Common Uses in VFX Work

Use Case

- Running Python scripts
- Launching applications
- Setting up environments
- Automating pipelines
- Copying/moving files

Description

- Call our **.py** files from a batch file
- Open tools like Maya, Nuke, Blender, etc., with specific arguments
- Set environment variables before launching software or scripts
- Chain together different stages of a VFX pipeline (e.g., **ingest** → **render**)
- Automate asset backup or directory cleanup

Running Python Scripts – in a .bat File

- Running Python Scripts on Terminal
- Running Python Scripts with a .bat File
- Handling Script Location Changes
- Using Python Script's Absolute Path
- Using Python Absolute Path
- Why Is This Useful in a Studio Environment?

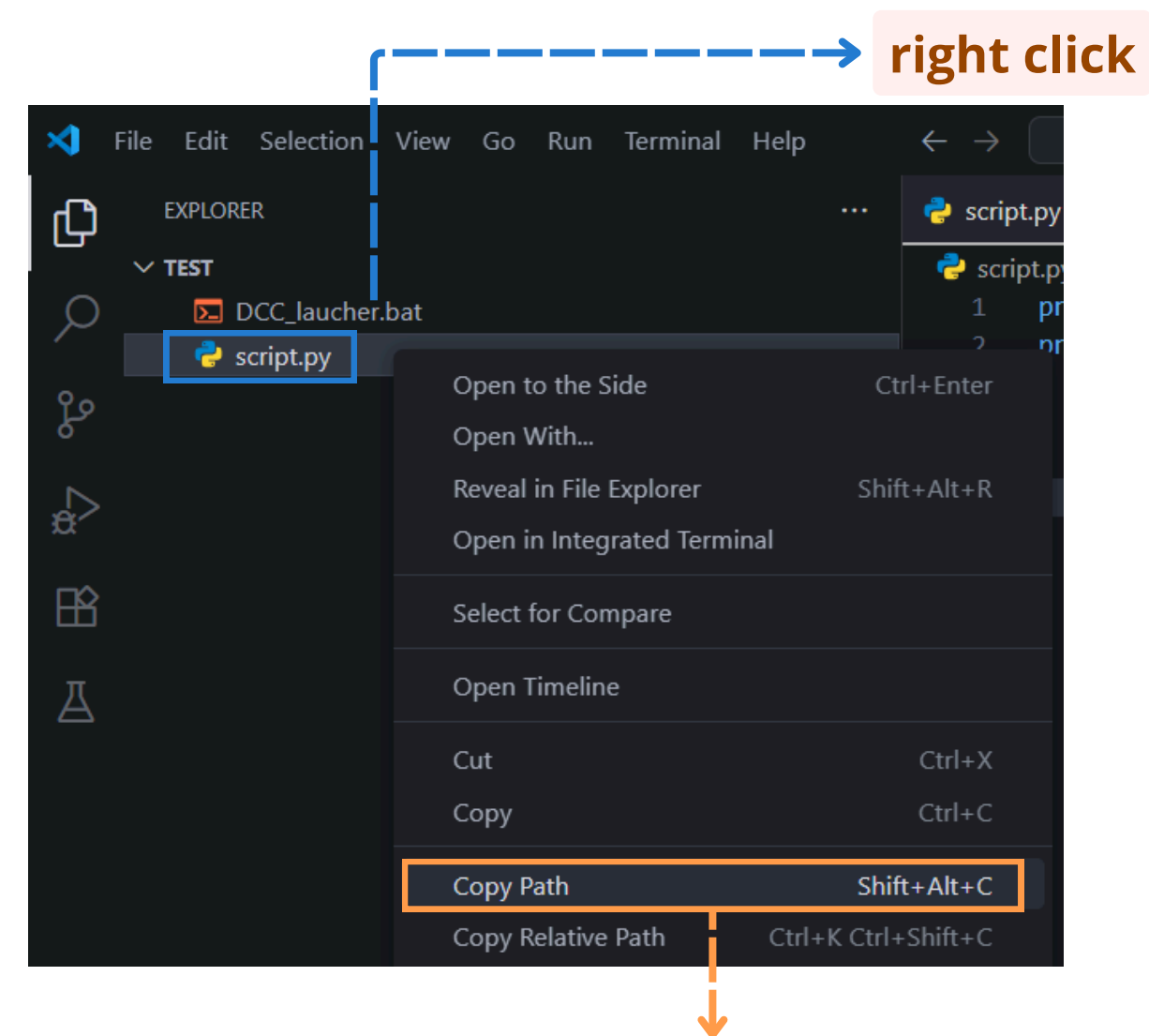
Running Python Scripts on Terminal

- Before we automate with a batch file, it's important to understand the **manual process**.
- This way, we'll know what the **.bat** file is actually doing behind the scenes.

Create a Python Script

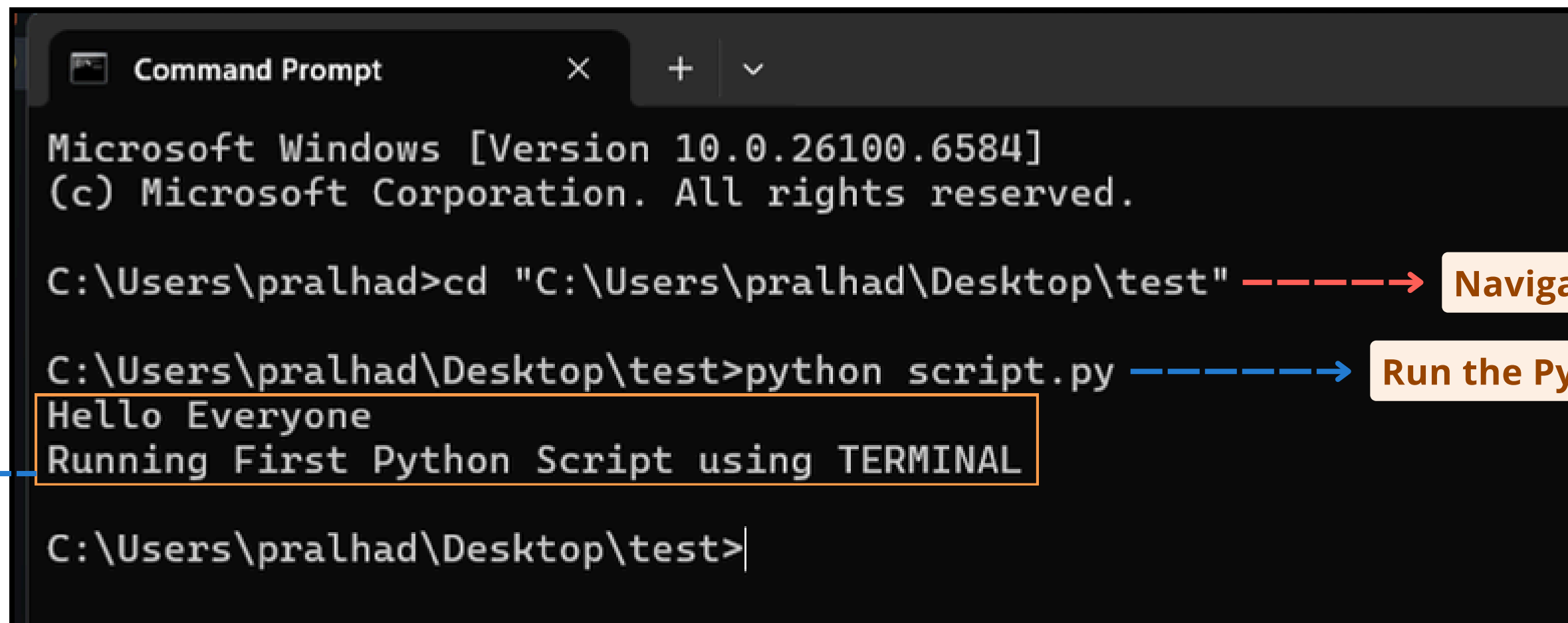
- Open a **text editor**
- Type the following code:

```
script.py
1 print("Hello Everyone")
2 print("Running First Python Script using TERMINAL")
3
```



Click to Copy Absolute path

Run the Script Manually



```
Command Prompt
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>cd "C:\Users\pralhad\Desktop\test"
C:\Users\pralhad\Desktop\test>python script.py
Hello Everyone
Running First Python Script using TERMINAL
C:\Users\pralhad\Desktop\test>
```

Navigate to the file location

Run the Python Script

Output

This **confirms** that our **script is working** and **Python** is set up **correctly**.

Running Python Scripts with a .bat File

- Let's **automate** this process using a **batch** file. This way, we don't need to open the **terminal manually** each time.
- We can just **double-click** the **.bat** file and **execute** the script.

Create the Batch File

Let's write a batch file that will run the **Python script** and display some text.

Folder Structure

Desktop



test.bat

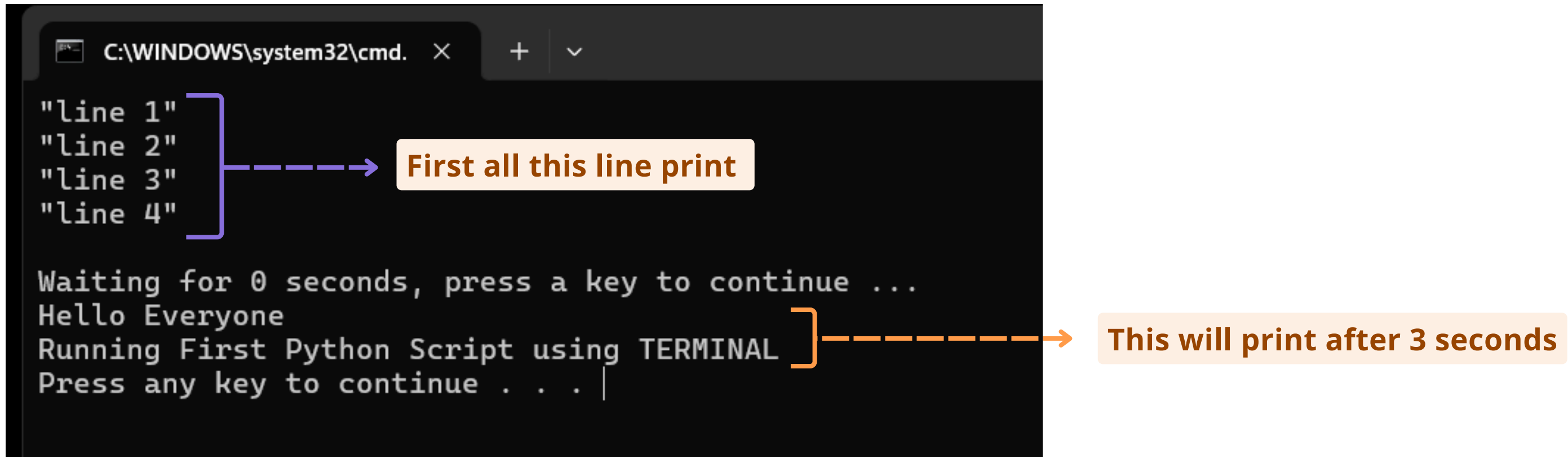
```
test.bat X
test.bat
1  @echo off
2  echo "line 1"
3  echo "line 2"
4  echo "line 3"
5  echo "line 4"
6  timeout 3
7
8  python script.py
9  pause
10
```

Runs our Python script

Make sure **both bat** file and **python file** should be in same directory

Run the Batch File

1. Go to your **Desktop**.
2. **Double-click** test.bat.



```
C:\WINDOWS\system32\cmd.
"line 1"
"line 2"
"line 3"
"line 4"
Waiting for 0 seconds, press a key to continue ...
Hello Everyone
Running First Python Script using TERMINAL
Press any key to continue . . . |
```

First all this line print

This will print after 3 seconds

Handling Script Location Changes

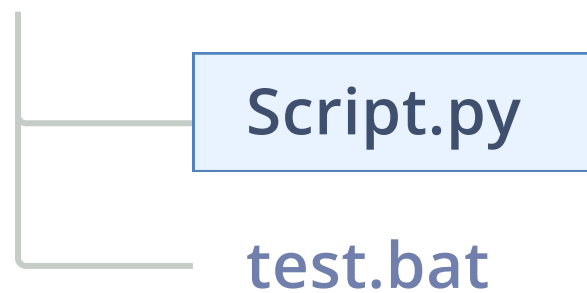


What Happens If We Move the Script.py File ?

Suppose we move our **Script.py** file from desktop location to somewhere else lets say inside a test folder

Original Location

Desktop/



Move py file inside test folder

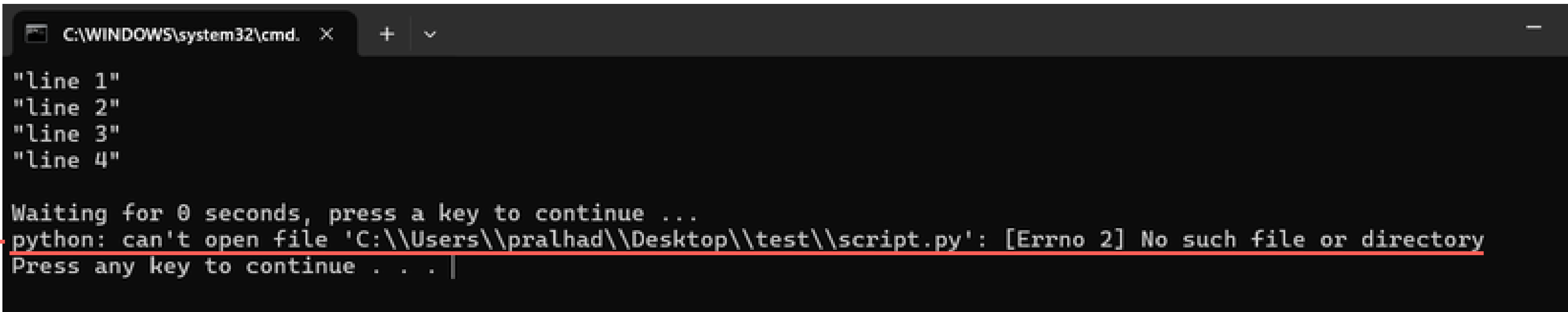
Desktop/



Now

- when we double-click our batch file (**test.bat**),
- it still looks for the script on the Desktop.

When we run file we get Error



```
C:\WINDOWS\system32\cmd. X + v

"line 1"
"line 2"
"line 3"
"line 4"

Waiting for 0 seconds, press a key to continue ...
python: can't open file 'C:\\Users\\pralhad\\Desktop\\test\\script.py': [Errno 2] No such file or directory
Press any key to continue . . . |
```

A red dashed line with an arrow points from the error message in the terminal to the 'This means:' section below.

This means:

- The **batch file** is still pointing to the **old path**.
- The **script doesn't exist** there anymore.

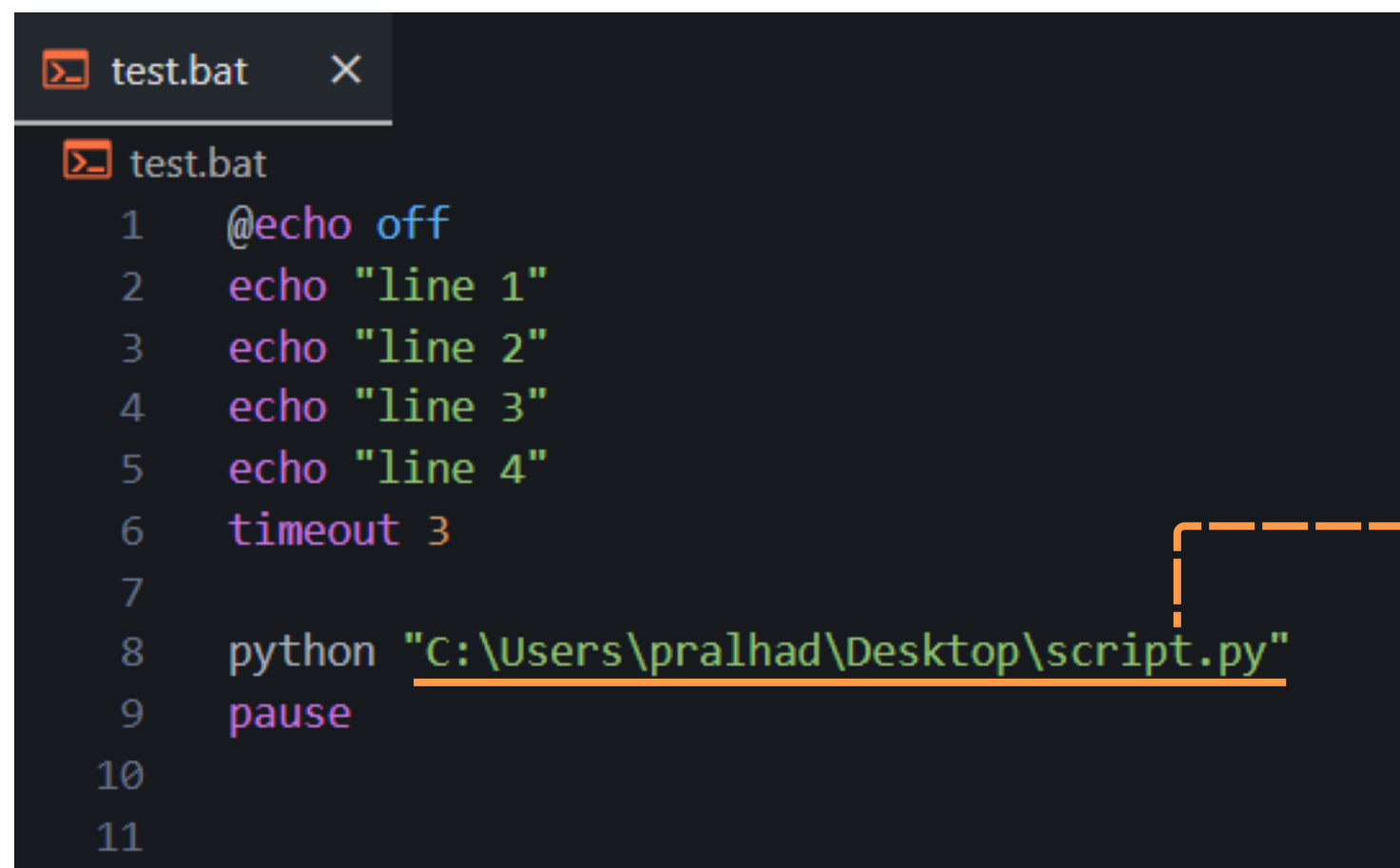
Why our Batch File Fails

- A **batch file** runs commands in the **current working directory** unless you give it a **full path**.
- Windows expects **script.py** to be in the same folder where the **batch file is being run**.

Using Python Script's Absolute Path

- To **avoid** this **problem**, we can use an **absolute path** for the **Python script**.
- This **ensures** that no matter where the script is, the **batch file** can still find and **execute it**.

Modify the Batch File



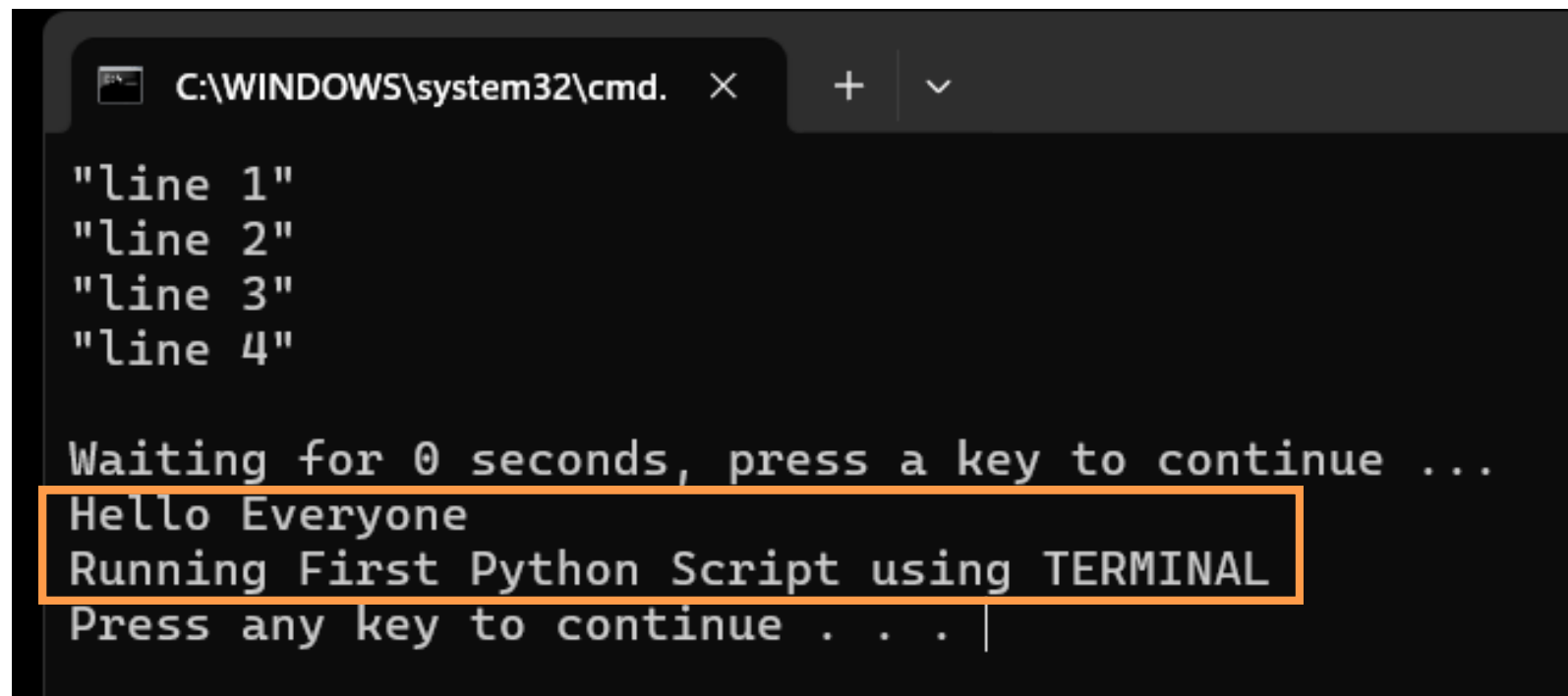
```
test.bat
1 @echo off
2 echo "line 1"
3 echo "line 2"
4 echo "line 3"
5 echo "line 4"
6 timeout 3
7
8 python "C:\Users\pralhad\Desktop\script.py"
9 pause
10
11
```

Absolute path of script.py file

Make sure to put the path in **double quotes** if it **contains spaces**.

Run the Batch File

Double-click **test.bat**.



A screenshot of a Windows Command Prompt window. The title bar shows the path "C:\WINDOWS\system32\cmd." with standard window controls. The command prompt displays the following text: "line 1", "line 2", "line 3", and "line 4" on separate lines. Below these, it says "Waiting for 0 seconds, press a key to continue ...". The next two lines, "Hello Everyone" and "Running First Python Script using TERMINAL", are highlighted with an orange rectangular box. The final line shown is "Press any key to continue . . . |", with a vertical cursor at the end.

```
C:\WINDOWS\system32\cmd.
"line 1"
"line 2"
"line 3"
"line 4"

Waiting for 0 seconds, press a key to continue ...
Hello Everyone
Running First Python Script using TERMINAL
Press any key to continue . . . |
```

Why Use an Absolute Path?

- Ensures the batch file can locate your Python script **from any folder**.
- Avoids "**File not found**" errors.
- Works even if the **.bat** file and **.py** file are in different locations.

Using Python Absolute Path

Sometimes the command **python** doesn't work in **Command Prompt** or inside a **batch file**.

This usually happens because:

- **Python isn't installed globally, or**
- Python is installed but **not added to the system's PATH ENV variable**.

```
'python' is not recognized as an internal or external command,  
operable program or batch file.
```

The Problem

When we write: → `python script.py`

- Windows assumes **two** things:
 - Python is installed on the machine.
 - The **python** command is available from any directory (added to **PATH**).

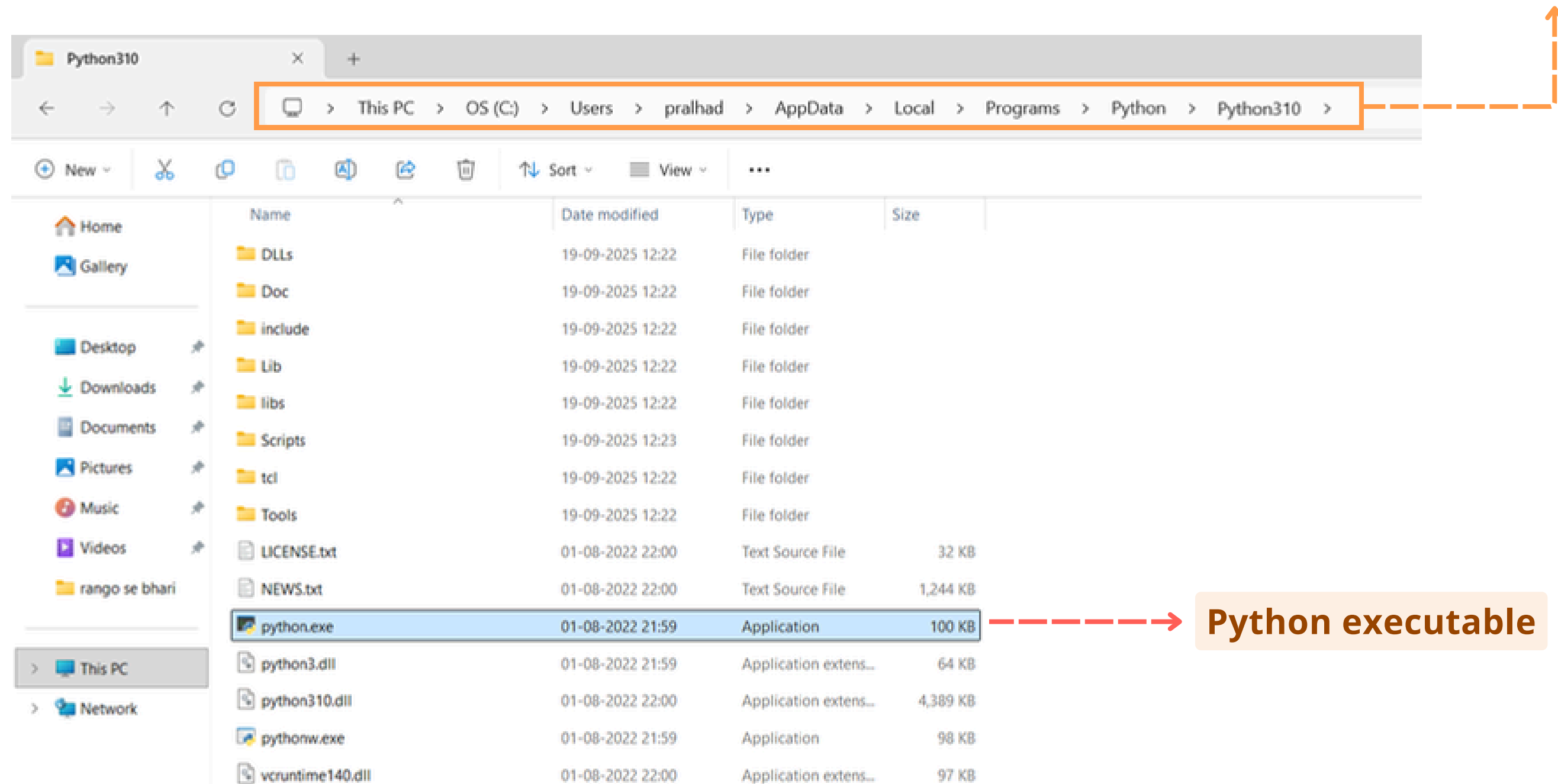
If either of these is false → the batch file fails.

The Solution: Use Python's Full Path

Instead of relying on **python**, call the **full path to the Python executable**.

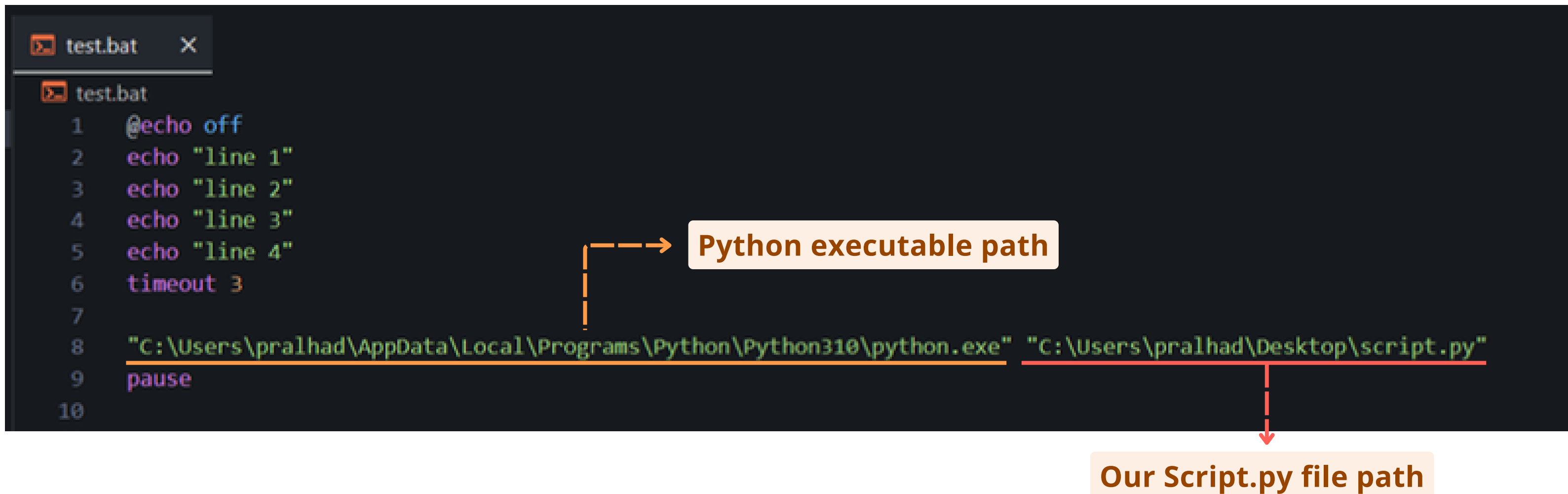
Find Python's Location

By Default python locate at : → `C:\Users\pralhad\AppData\Local\Programs\Python\Python310\python.exe`



Update the Batch File

Modify your **.bat** file to reference Python directly:



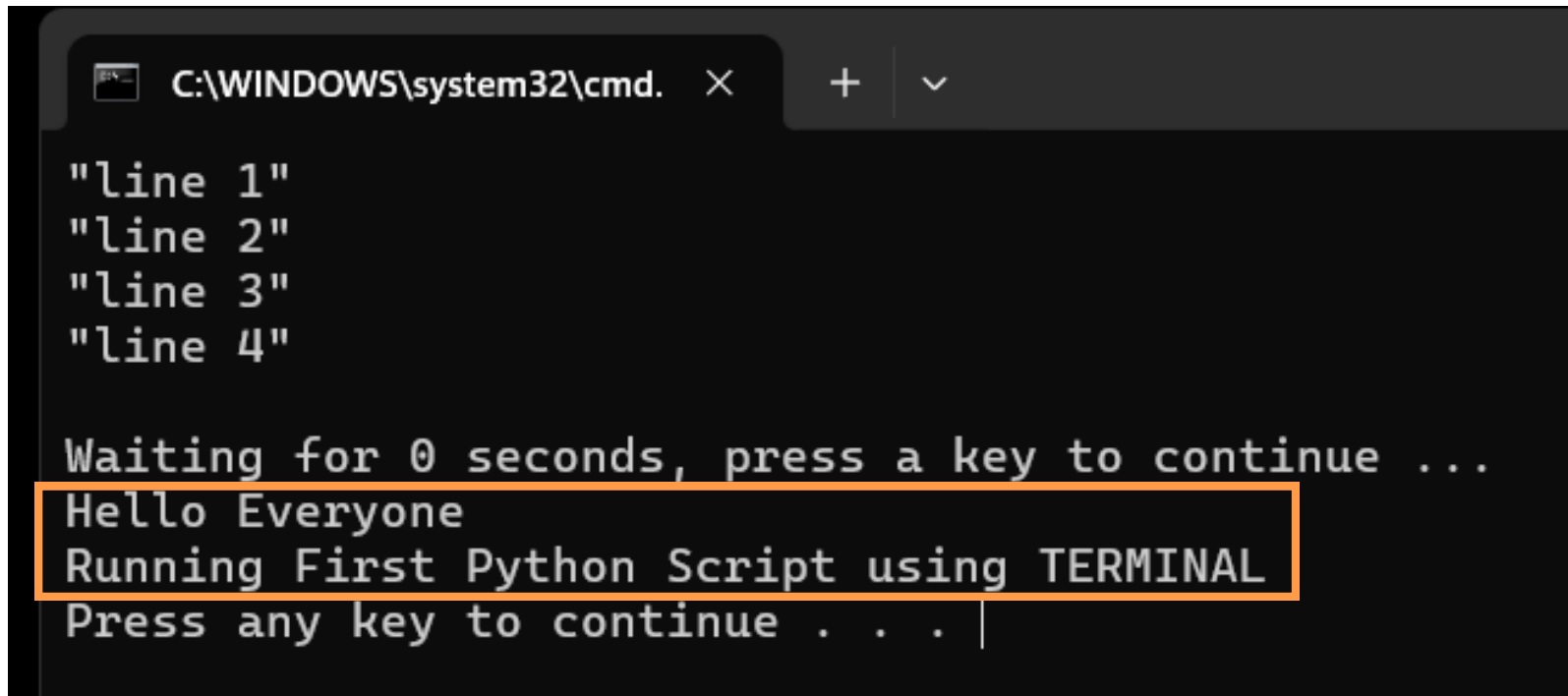
```
test.bat X
test.bat
1 @echo off
2 echo "line 1"
3 echo "line 2"
4 echo "line 3"
5 echo "line 4"
6 timeout 3
7
8 "C:\Users\pralhad\AppData\Local\Programs\Python\Python310\python.exe" "C:\Users\pralhad\Desktop\script.py"
9 pause
10
```

Python executable path

Our Script.py file path

Run the Batch File

Double-click **test.bat**.

A screenshot of a Windows Command Prompt window. The title bar shows the path 'C:\WINDOWS\system32\cmd.' and standard window controls. The command prompt displays the following text: '"line 1"', '"line 2"', '"line 3"', '"line 4"', 'Waiting for 0 seconds, press a key to continue ...', 'Hello Everyone', 'Running First Python Script using TERMINAL', and 'Press any key to continue . . . |'. The last three lines are enclosed in an orange rectangular box.

```
C:\WINDOWS\system32\cmd.
"line 1"
"line 2"
"line 3"
"line 4"
Waiting for 0 seconds, press a key to continue ...
Hello Everyone
Running First Python Script using TERMINAL
Press any key to continue . . . |
```

Why This Works

- The batch file no longer depends on the global python command.
- It will run Python from the exact location we specify.
- It's more reliable, especially when sharing scripts with others.

Why Is This Useful in a Studio Environment?

In a VFX studio, it's common to have:

- Multiple machines with similar setups
- Python installed in a standard location on every system

By using the full path to Python:

- We **avoid issues with missing environment variables**
- We ensure that the same version of Python is used across all systems
- Our batch file works the same on every workstation without extra setup

So far, we've explored **Four Main Approaches** to Run a .bat file

Manual Execution

- We first ran the Python script manually using the terminal with `python script.py`.

Basic Batch File

- Then we created a simple .bat file that runs the script using just `python script.py`.
- This only works if the batch file and Python script are in the same folder.

Absolute Script Path

- Next, we updated the batch file to use the full path to the Python script.
- So it runs correctly no matter where the .bat file is located.

Full Python Executable + Script Path

- Finally, we used the absolute path to both the `python.exe` and the script file.
- This is useful when Python is not added to the system environment variables
- The batch file will still run successfully.

Running Python Script – with Arguments

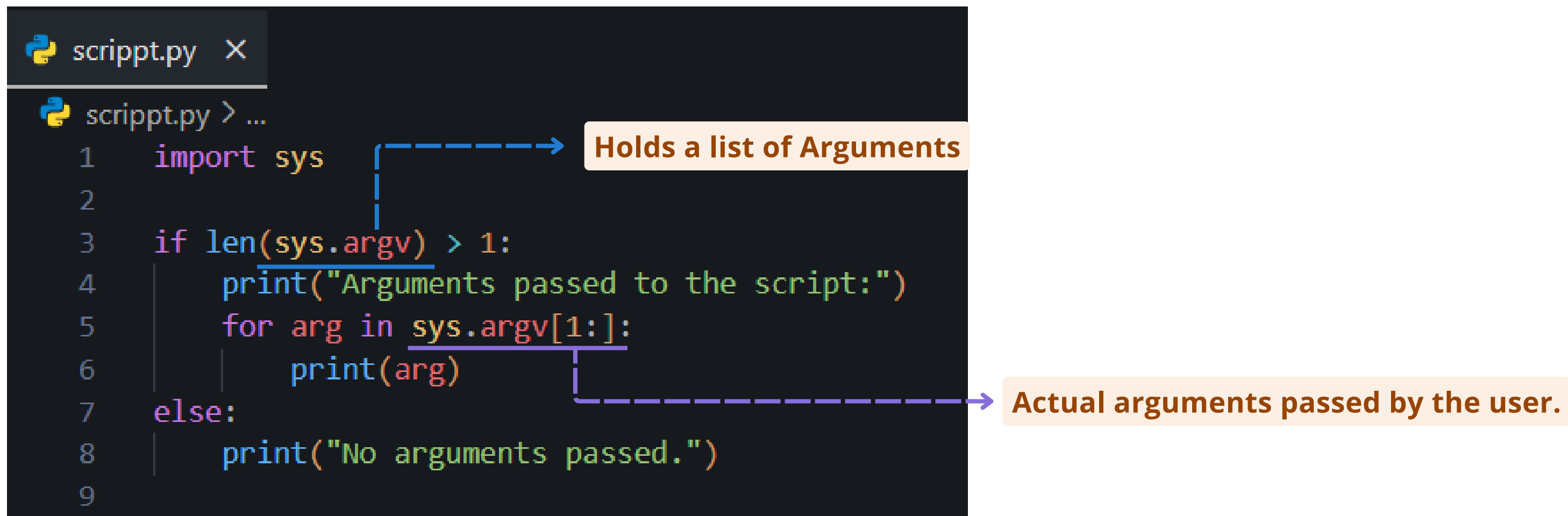
- Using Arguments
- Why Use Arguments in Batch Files?
- Practical Use Case: Automating Shot Processing

Using Arguments

Arguments are extra values you pass to a script when running it from the terminal or a **.bat** file.

- These values let you make scripts more **flexible** and **reusable**.
- In Python, arguments are accessed through the **sys.argv** list.

Code: script.py



```
scrippt.py X
scrippt.py > ...
1  import sys
2
3  if len(sys.argv) > 1:
4      print("Arguments passed to the script:")
5      for arg in sys.argv[1:]:
6          print(arg)
7  else:
8      print("No arguments passed.")
9
```

Annotations:

- Holds a list of Arguments**: Points to `sys.argv` in line 3.
- Actual arguments passed by the user.**: Points to `sys.argv[1:]` in line 5.

Save this script as **script.py** in: → `C:\Users\pralhad\Desktop\script.py`

Create the Batch File

Now let's write a batch file that will run the script **with arguments**.

Code: test.bat



```
test.bat X
test.bat
1 @echo off
2 "C:\Users\pralhad\AppData\Local\Programs\Python\Python310\python.exe" "C:\Users\pralhad\Desktop\script.py" arg1 arg2
3 pause
4
5
```

full path to Python executable.

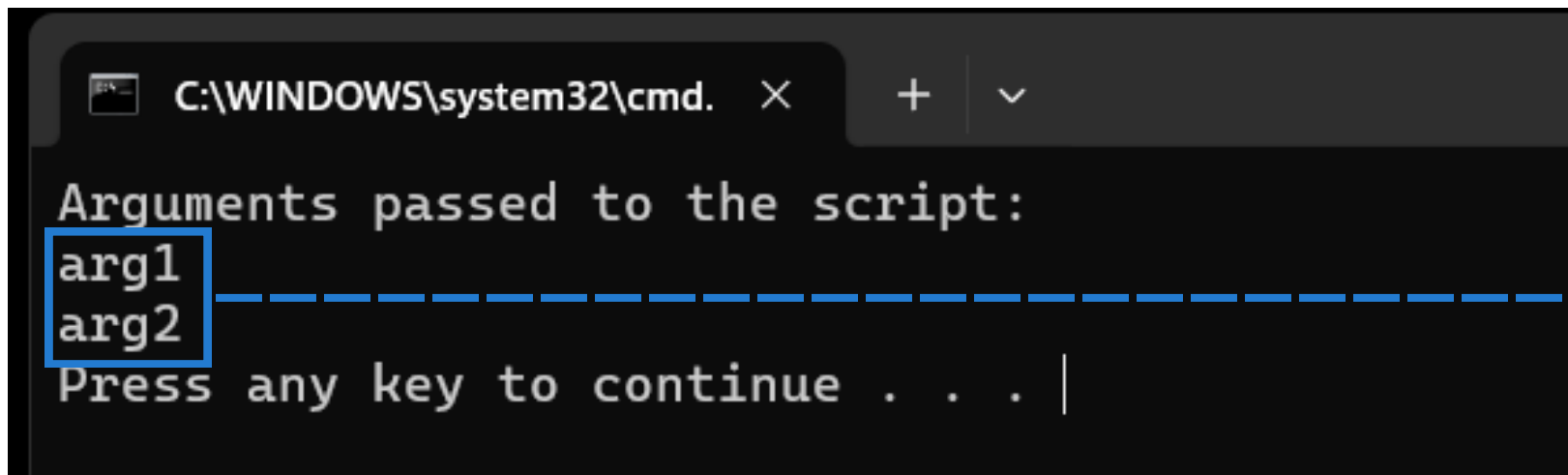
full path to the script.

arguments being passed.

Make sure the paths to **python.exe** and **script.py** match the **actual locations** on **your system**.

Run the Batch File

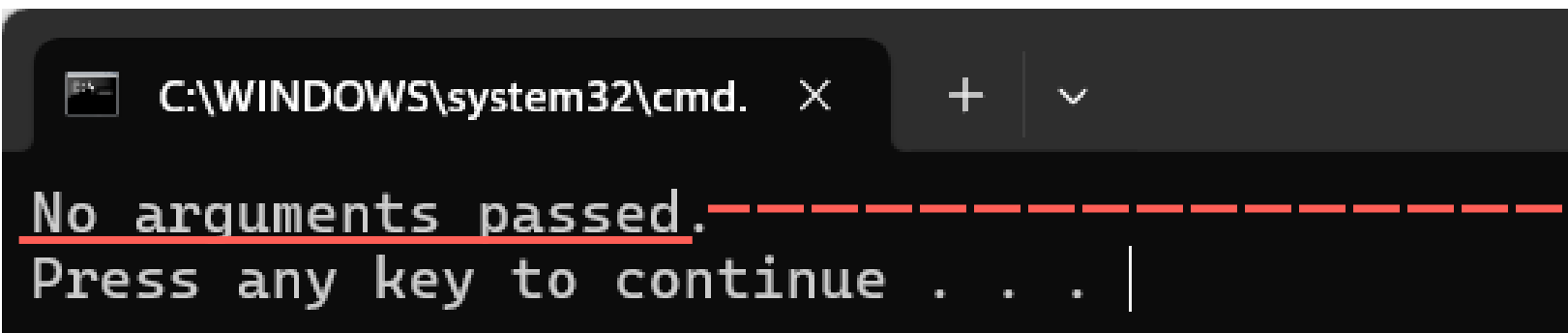
- Double-click **test.bat**.
- The terminal will open and run your script with arguments.



A screenshot of a Windows command prompt window. The title bar shows 'C:\WINDOWS\system32\cmd.' with a close button and window controls. The text inside the window reads: 'Arguments passed to the script:', followed by 'arg1' and 'arg2' on separate lines, which are enclosed in a blue rectangular box. Below this, it says 'Press any key to continue . . . |'.

actual arguments passed to the script

If you remove the arguments (**arg1 arg2**), the output will be:



A screenshot of a Windows command prompt window. The title bar shows 'C:\WINDOWS\system32\cmd.' with a close button and window controls. The text inside the window reads: 'No arguments passed.', followed by 'Press any key to continue . . . |'.

Printed when no arguments were provided

✓ **Result:** You've now automated a Python script that accepts arguments **using a batch file**.



What's the purpose of passing arguments to a bat file?

Why Use Arguments in Batch Files?

The Problem

In production, we often deal with **multiple versions of the same software** or **different configurations** for different projects. **For example:**

Projects	Requirements	For Render
• Project A	→ Nuke Version 14.0	→ Frame Range 1 - 100
• Project B	→ Nuke Version 13.2 + Special Config File	→ Frame Range 101 - 200

Without arguments, you'd need separate batch files for each variation → **messy, harder** to **maintain**.

The Solution: Use Arguments

By passing arguments to a batch file, we can use a **single script** and make it **adaptable** for different needs.

Examples

Here are a few examples of how arguments make the **batch file flexible**:

1. Launch a Specific Version:

```
test.bat version2024 config.cfg
```

tells the script to launch Maya 2024.

provides a configuration file.

2. Run a Render with Parameters:

```
test.bat render_scene frame_start=1 frame_end=100
```

task to perform.

control the frame range

This helps when working with:

- Different **software versions**
- Different **configuration files**
- Custom **parameters** like frame ranges, resolution, or paths

Why It Matters in VFX Workflows

- **Flexibility** → Switch between tools, versions, or configs quickly.
 - **Reusability** → Maintain one script instead of dozens.
 - **Scalability** → Handle different projects or shots without editing the script itself.
- 

Practical Use Case: Automating Shot Processing

In a VFX studio, you may need to process multiple shots for tasks like:

- **Rendering** a specific shot.
- Applying **color grading** presets.
- Generating **preview videos** for client approval.

Instead of editing the script every time, you can pass the shot name and task as arguments via a batch file.



Automating Shot Processing

process_shot.py

```
import sys

if len(sys.argv) != 3:
    print("Usage: process_shot.py <shot_name> <task>")
    print("Example tasks: render, color_grading, preview")
    sys.exit(1)

shot_name = sys.argv[1]
task = sys.argv[2]

if task == "render":
    print(f"Rendering Shot: {shot_name}")
elif task == "color_grading":
    print(f"Applying color grading to Shot: {shot_name}")
elif task == "preview":
    print(f"Generating preview for Shot: {shot_name}")
else:
    print(f"Unknown task '{task}'. Supported tasks: render, color_grading, preview")
    sys.exit(1)

print("Task completed successfully!")
```

Checks if the user passed exactly 2 arguments

The shot name (e.g. shot_001)

task to perform (e.g. render, color_grading)

Executes logic depending on the task passed

- "render" → Simulates rendering the shot
- "color_grading" → Simulates applying color grading
- "preview" → Simulates generating a preview

Catches unsupported tasks

Printed only if a valid task was executed.

Create the Batch File (run_shot.bat)



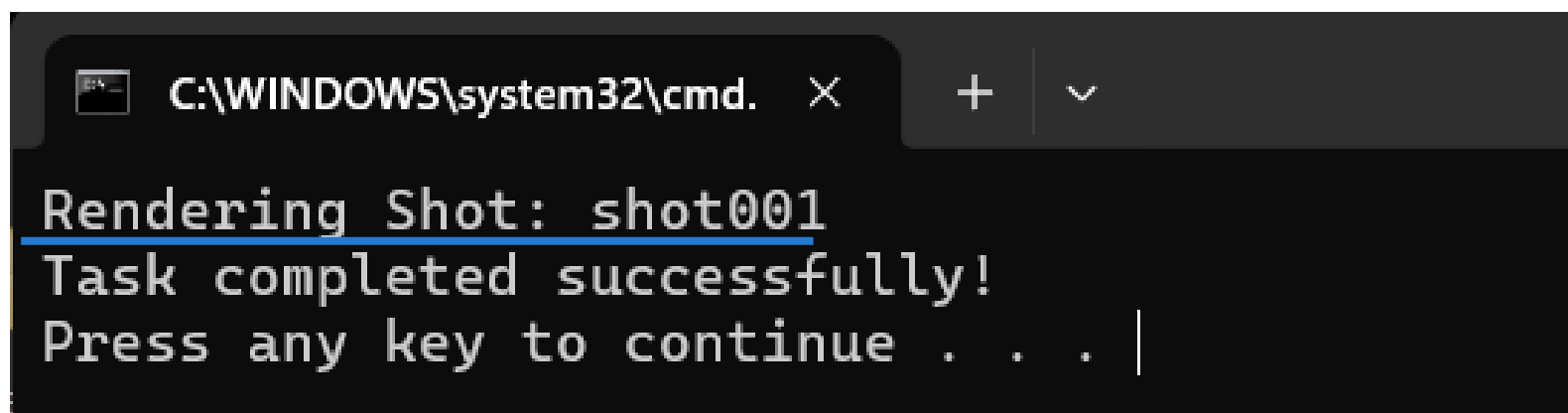
```
run_shot.bat X
run_shot.bat
1 @echo off
2 "C:\Users\pralhad\AppData\Local\Programs\Python\Python310\python.exe" "C:\Users\pralhad\Desktop\test\process_shot.py" shot001 render
3 pause
4
5
```

name of the shot to process.

task to perform on the shot.

Run the Batch File

Double-click **run_shot.bat**. The output will be:



```
C:\WINDOWS\system32\cmd. X + v
Rendering Shot: shot001
Task completed successfully!
Press any key to continue . . . |
```

For color grading:

```
run_shot.bat
1 @echo off
2 "C:\Users\pralhad\AppData\Local\Programs\Python\Python310\python.exe" "C:\Users\pralhad\Desktop\test\process_shot.py" shot001 color_grading
3 pause
4
```

Output:

```
C:\WINDOWS\system32\cmd. x + v
Applying color grading to Shot: shot001
Task completed successfully!
Press any key to continue . . . |
```

For preview generation:

```
run_shot.bat
1 @echo off
2 "C:\Users\pralhad\AppData\Local\Programs\Python\Python310\python.exe" "C:\Users\pralhad\Desktop\test\process_shot.py" shot001 preview
3 pause
4
```

Output:

```
C:\WINDOWS\system32\cmd. x + v
Generating preview for Shot: shot001
Task completed successfully!
Press any key to continue . . . |
```


Why This Approach Is Useful in VFX Pipelines?

- Flexible: Run different tasks without modifying the script.
- Organized: Easily switch between shots and tasks via the batch file.
- Efficient: Save time by automating repetitive tasks.
- Scalable: Process multiple shots by creating multiple batch files or running them in loops.

This approach keeps our workflow clean, scalable, and easy to manage - ideal for busy production environments.



Running Python Scripts – Using Relative Paths

- Why Start with Absolute Paths?
- Why Use Relative Paths?
- Absolute vs. Relative Path
- Using Relative Paths
- Understanding %~dp0
- Why Use %~dp0 in Batch Files?
- Best Practice: Always Store Paths in Variables

Why Start with Absolute Paths?

When learning, it's **common to use absolute paths**, for example:



```
C:\Python39\python.exe  
C:\Projects\myscript.py
```

path of python executable

path of script

- Easy to understand — they tell Windows the exact location of the files.
- Useful for quick tests on your own machine.

But there's a downside:

- ✗ They only work on your computer.
- ✗ If another artist has Python or the project in a different folder, the script will fail.

Why Use Relative Paths?

In real **VFX** production, scripts and tools aren't just used on one computer — they're shared across:

- **Multiple artists** working on the same project.
- **Different machines** with different setups.
- **Network drives** where files move between departments.

If you **hardcode absolute paths**, the script will **often break** when run on another workstation.

✓ **Relative paths solve this problem.**

- They make scripts **portable** → the same .bat file works anywhere.
- As long as the **folder structure stays consistent**, the script will run without modification.

Absolute vs. Relative Path

◆ Absolute Path (Fixed Location)

```
"C:\Python39\python.exe" "C:\Projects\myscript.py"
```

- Always points to one **exact location** on disk.
- ✗ Works **only** if Python and the script exist in those paths.
- ✗ Breaks easily when moving **between different machines**.

◆ Relative Path (Flexible Location)

```
python "%~dp0\myscript.py"
```

- **%~dp0** → expands to the folder where the batch file is located.
- ✓ Works as long as the batch file and script are in the **same folder** (or consistent subfolders).
- ✓ Much **more portable** — ideal for shared VFX project setups.

Using Relative Paths

Folder Structure Example

Imagine you have the following **folder structure**:

```
ProjectFolder/  
├── bin/  
│   ├── run_script.bat  
│   └── myscript.py
```

myscript.py

```
print("Hello from myscript.py!")  
print("This script is running successfully using a relative path.")
```

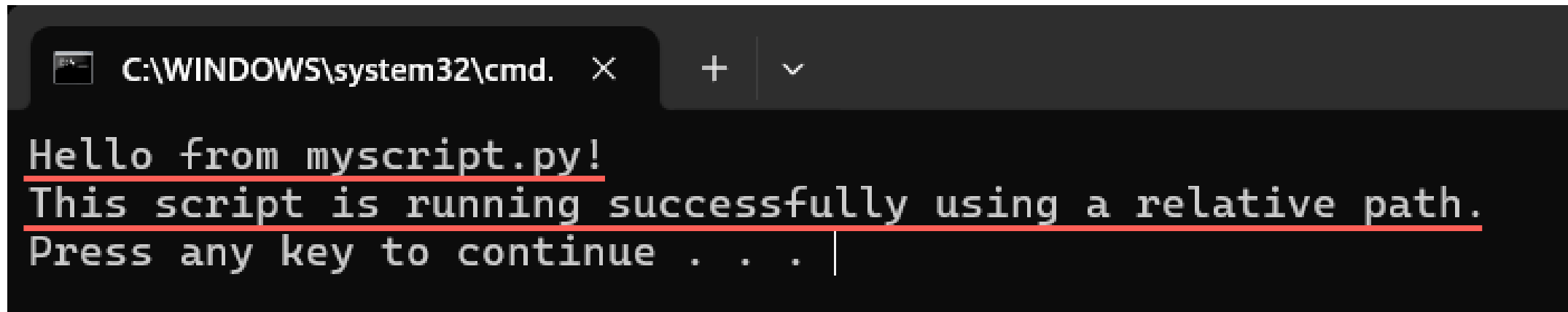
run_script.bat

```
> run_script.bat  
@echo off  
  
python "%~dp0\myscript.py"  
pause
```

Refer to Current Folder

Run the Batch File

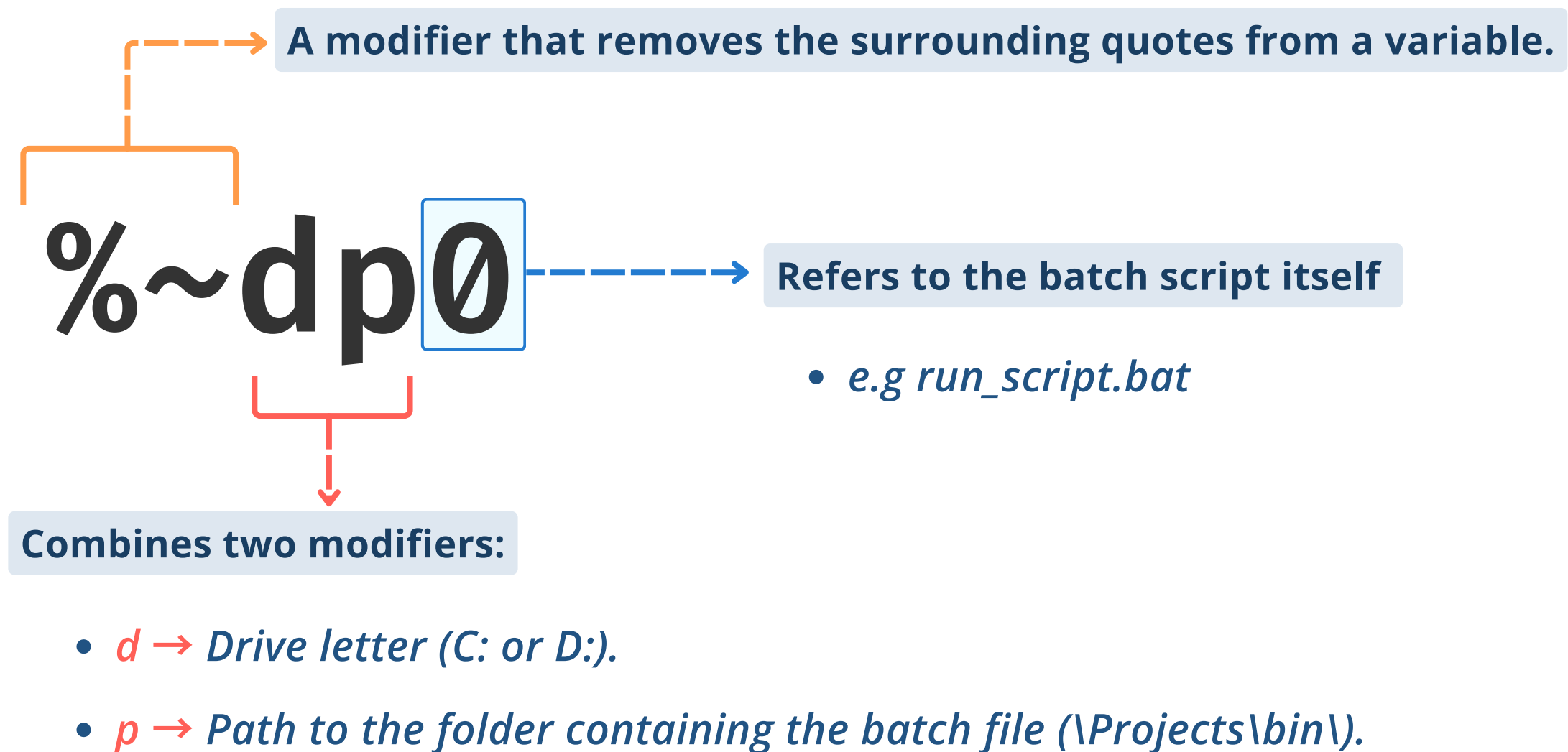
- Double-click **run_script.bat**. If set up correctly, it will execute the script using the **Python interpreter**.



```
C:\WINDOWS\system32\cmd.  X  +  v  
Hello from myscript.py!  
This script is running successfully using a relative path.  
Press any key to continue . . . |
```

Understanding %~dp0

- %~dp0 is a **special variable** that **expands to the full drive and path of the folder** where the **batch file is located**.
- It's one of the most **powerful tools** for **handling relative paths, especially in portable scripts**.



| **%~dp0** → The full path of the folder where the batch file is located, including the drive letter.

Displaying %~dp0 in Action

Let's create a **simple batch** file to display the value of **%~dp0**

Folder Structure

Desktop/

test/

show_path.bat

```
show_path.bat X
> show_path.bat
1 @echo off
2
3 set batchFolder=%~dp0
4 echo The batch file is located in: %batchFolder%
5
6 pause
```

Stores the path

Drive + Path of the batch file location

replaced with the actual folder location.

Output

```
C:\WINDOWS\system32\cmd. X + v
The batch file is located in: C:\Users\pralhad\Desktop\test\
Press any key to continue . . .
```

Bat file Full Path

Accessing Files Relative to the Batch File

Imagine the following folder structure:

```
C:\Projects\  
├── bin\  
│   ├── run_script.bat  
│   ├── myscript.py  
├── assets\  
│   └── texture.jpg
```

```
run_script.bat X  
bin > run_script.bat  
1 @echo off  
2 set batchFolder=%~dp0  
3 echo Batch file is in: %batchFolder%  
4  
5 echo Accessing Python Script:  
6 python "%batchFolder%myscript.py"  
7  
8 echo Accessing Asset File:  
9 echo Asset path: "%batchFolder%..\assets\texture.jpg"  
10  
11 pause
```

Store bat file full path

Run Script located to same folder

Shows the relative path to texture.jpg

Output

```
C:\WINDOWS\system32\cmd. X + v  
Batch file is in: C:\Users\pralhad\Desktop\test\bin\  
Accessing Python Script:  
Hello from myscript.py!  
This script is running successfully using a relative path.  
Accessing Asset File:  
Asset path: "C:\Users\pralhad\Desktop\test\bin\..\assets\texture.jpg"  
Press any key to continue . . .
```

Full path of bat file

Output of python file

path of image

Why Use %~dp0 in Batch Files?

Portability:

- The batch file works regardless of where the project folder is moved.

Reliability:

- Avoids hardcoding absolute paths that may not exist on other systems.

Environment Independence:

- Works seamlessly across different environments (e.g., local development, staging, production).

Reduced Errors

- Absolute paths can cause errors if the exact path doesn't exist on another machine.

Flexibility:

- You can easily reference files in subfolders or parent folders using relative paths.

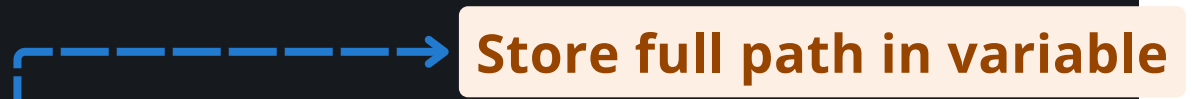
Automation-Friendly:

- Perfect for use in scripts deployed across multiple environments.

Best Practice: Always Store Paths in Variables

When using **%~dp0**, store it in a variable if you **need to reference it multiple times**:

```
run_script.bat X
bin > run_script.bat
1  @echo off
2
3  set basePath=%~dp0
4
5  echo Base Path: %basePath%
6  python "%basePath%\myscript.py"
7
8  echo Access asset: "%basePath%\..\assets\texture.jpg"
9  pause
10
```



Benefits

- **Cleaner scripts** → no need to repeat %~dp0 everywhere.
- **Easier to debug** → print the variable once, check if the path is right.
- **More readable** → anyone reading the file quickly sees where the base path comes from.

Number Range Validator

Create a .bat file to open the Calculator application and display a custom message.

```
@echo off
echo Launching Calculator...
timeout 2
start "" "C:\Windows\System32\calc.exe"
pause
```

Batch File Rename Simulator

Write a .bat file that opens a specific project folder and Notepad in separate windows.

```
@echo off
echo Setting up your workspace...
start "" "C:\Users\%USERNAME%\Documents\MyProject"
start "" "C:\Windows\System32\notepad.exe"
pause
```

Launch Multiple Tools

Create a .bat file that launches Paint, WordPad, and the Command Prompt, with a countdown before starting.

```
@echo off
echo Preparing to launch tools...
timeout 3
start "" "C:\Windows\System32\mspaint.exe"
start "" "C:\Program Files\Windows NT\Accessories\wordpad.exe"
start "" "C:\Windows\System32\cmd.exe"
echo All tools launched successfully!
pause
```


Run Script (Relative Path)

Create a .bat file to run a Python script saved in the same folder as the batch file.

```
@echo off
echo Running Python script from the current folder...
python script.py
pause
```

Run Script (Absolute Path)

Create a .bat file to run a Python script using its absolute path.

```
@echo off
echo Running Python script from an absolute path...
python "C:\Users\%USERNAME%\Documents\myscript.py"
pause
```

Add Countdown Before Script

Create a .bat file that displays a countdown before executing a Python script.

```
@echo off
echo Preparing to run the Python script...
timeout 3
echo 3
timeout 1
echo 2
timeout 1
echo 1
timeout 1
python "C:\Users\%USERNAME%\Documents\countdown_script.py"
pause
```

Handle Missing Python Path

Create a .bat file that runs a Python script using the Python executable's absolute path and adds a message if the executable is missing.

```
@echo off
if exist "D:\Python3_9_7\python.exe" (
    echo Python executable found. Running the script...
    "D:\Python3_9_7\python.exe" "C:\Users\%USERNAME%\Desktop\example_script.py"
) else (
    echo Python executable not found at the specified path.
)
pause
```


Run Script with Argument

Create a .bat file to pass a single argument to a Python script that prints a custom message.

Python Script (single_arg.py):

```
import sys
if len(sys.argv) > 1:
    print(f"Hello, {sys.argv[1]}!")
else:
    print("No name provided.")
```

Batch File (run_single_arg.bat):

```
@echo off
"D:\Python3_9_7\python.exe" C:\Users\%USERNAME%\Desktop\single_arg.py Student
pause
```

Automate Shot Render

Create a .bat file to pass a shot name and task (render or preview) to a Python script.

Python Script (render_shot.py):

```
import sys
if len(sys.argv) == 3:
    shot = sys.argv[1]
    task = sys.argv[2]
    if task == "render":
        print(f"Rendering shot: {shot}")
    elif task == "preview":
        print(f"Generating preview for shot: {shot}")
    else:
        print(f"Unknown task: {task}. Use 'render' or 'preview'.")
else:
    print("Usage: render_shot.py <shot> <task>")
```

Batch File (run_render.bat):

```
@echo off
"D:\Python3_9_7\python.exe" C:\VFX_Tools\render_shot.py shot001 render
pause
```

Pass Frame Range

Create a .bat file to pass a frame range (start and end) to a Python script and display the frames being processed.

Python Script (frame_range.py):

```
import sys
if len(sys.argv) == 3:
    start_frame = int(sys.argv[1])
    end_frame = int(sys.argv[2])
    print(f"Processing frames from {start_frame} to {end_frame}...")
else:
    print("Usage: frame_range.py <start_frame> <end_frame>")
```

Batch File (run_frames.bat):

```
@echo off
"D:\Python3_9_7\python.exe" C:\Users\%USERNAME%\Desktop\frame_range.py 1 100
pause
```

Run Script (Same Folder)

Create a .bat file to run a Python script located in the same folder using relative paths.

Folder Structure:

```
ProjectFolder/
├── run_script.bat
└── myscript.py
```

File Content:

- myscript.py

```
print("Hello from myscript.py!")
```

- run_script.bat

```
@echo off
python "%~dp0myscript.py"
pause
```


Display Subfolder File

Create a .bat file to display the contents of a file located in a subfolder relative to the batch file's location.

Folder Structure:

```
ProjectFolder/  
├── run_script.bat  
├── data/  
│   └── info.txt
```

File Content:

- data/info.txt

```
This is a sample text file in the data folder.
```

- run_script.bat

```
@echo off  
type "%~dp0data\info.txt"  
pause
```

Access Parent Folder File

Create a .bat file to run a Python script and pass a configuration file from the parent folder as an argument.

Folder Structure:

```
ProjectFolder/  
├── config.cfg  
├── scripts/  
│   ├── run_script.bat  
│   └── myscript.py
```

File Content:

- config.cfg
- scripts/myscript.py

```
[Settings]  
Option1=True  
Option2=False
```

```
import sys  
config_path = sys.argv[1]  
print(f"Reading configuration from: {config_path}")
```

- scripts/run_script.bat

```
@echo off  
python "%~dp0myscript.py" "%~dp0..\config.cfg"  
pause
```

Run Multiple Scripts

Automate running multiple Python scripts located in the same folder as the batch file.

Folder Structure:

```
ProjectFolder/  
├─ run_pipeline.bat  
├─ prepare_assets.py  
├─ render_scene.py  
└─ generate_preview.py
```

File Content:

- prepare_assets.py

```
print("Step 1: Preparing assets...")
```

- render_scene.py

```
print("Step 2: Rendering scene...")
```

- generate_preview.py

```
print("Step 3: Generating preview...")
```

- run_pipeline.bat

```
@echo off  
python "%~dp0prepare_assets.py"  
python "%~dp0render_scene.py"  
python "%~dp0generate_preview.py"  
pause
```

Run Script Using %~dp0

Create a .bat file that checks if a Python script exists in the same folder and runs it if available.

Folder Structure:

```
ProjectFolder/  
├─ run_script.bat  
├─ myscript.py
```

File Content:

- prepare_assets.py

```
print("Hello! This script was found and executed successfully.")
```

- render_scene.py

```
@echo off  
if exist "%~dp0myscript.py" (  
    echo Found myscript.py. Running now...  
    python "%~dp0myscript.py"  
) else (  
    echo Error: myscript.py not found in the current directory.  
)  
pause
```

Access Sibling Folder File

Create a .bat file to run a Python script and access a text file from a sibling folder.

Folder Structure:

```
ProjectFolder/  
├── scripts/  
│   ├── run_script.bat  
│   └── myscript.py  
└── data/  
    └── sample.txt
```

File Content:

- data/sample.txt

```
This is sample data for processing.
```

- scripts/myscript.py

```
import sys  
file_path = sys.argv[1]  
with open(file_path, 'r') as file:  
    print("Content of the file:")  
    print(file.read())
```

- scripts/run_script.bat

```
@echo off  
python "%~dp0myscript.py" "%~dp0..\data\sample.txt"  
pause
```

Thank You

